Advanced Computer Graphics (ACG)

Theory Session 5 – Indirect Illumination and Path Tracing

2026-2027

Pompeu Fabra University

Ricardo Marques (ricardo.marques@upf.edu)

Calendar

Week no.	Date	Theory	Practice	Seminar	Deliveries
Week 1	28 Sep 02 Oct	**T1**: Introduction	**P1**: Introduction to A1.1	-	
Week 2	05 Oct 09 Oct	**T2**: Basic Ray Tracing		**S1**: Work on A1	
Week 3	12 Oct 16 Oct	**T3**: Ray Casting, Recursive Ray Tracing	**P2**: Introduction to A1.2		
Week 4	19 Oct 23 Oct	**T4**: Rendering Equation, Monte Carlo Integration and Direct Illumination **T5**: Indirect Illumination and Path Tracing		**S2**: Work on A1	**D.A1**
Week 5	26 Oct 30 Oct	**T6**: Volume Rendering: Absorption-Only Model	**P3**: Introduction to A2.1		
Week 6	02 Nov 06 Nov	**T7**: Volume Rendering: Emission, Absorption and Scattering Model **T8:** Neural Radiance Fields and Topics' Presentation		**S3**: Work on A2	
Week 7	09 Nov 13 Nov	**T9**: **Midterm**	**P4**: Introduction to A2.2		
Week 8	16 Nov 20 Nov		**P5**: Work on A2	**S4**: Work on A2	**D.A2**
Week 9	23 Nov 27 Nov	**T10**: **1st Presentation**	**P6**: Work on FP	-	
Week 10	30 Nov 04 Dec	**T11**: **2nd Presentation** **(First Session)** **T12**: **2nd Presentation** **(Second Session)**		**S5**: Work on FP	**D.FP**

Class Outline

- Part I – Path Tracing
  - Indirect Illumination
  - Path-based perspective of the Global Illumination problem
  - Path Tracer V0
  - Path Tracer V1
  - Path Tracing with Next Event Estimation
- Part II – Advanced Path Tracing
  - Importance Sampling for Path Tracing
  - Links to Machine Learning



Part I
Path Tracing

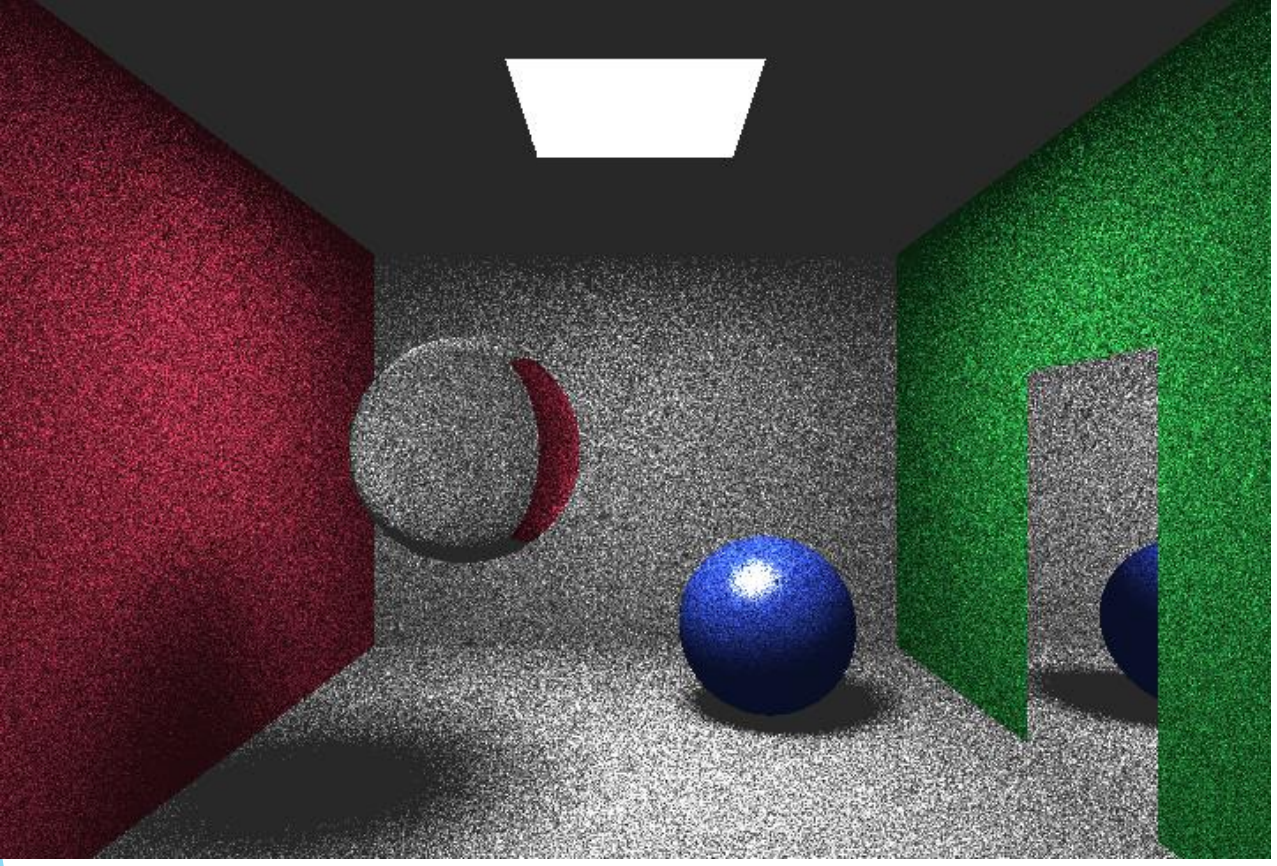
Last Class: Direct Illumination

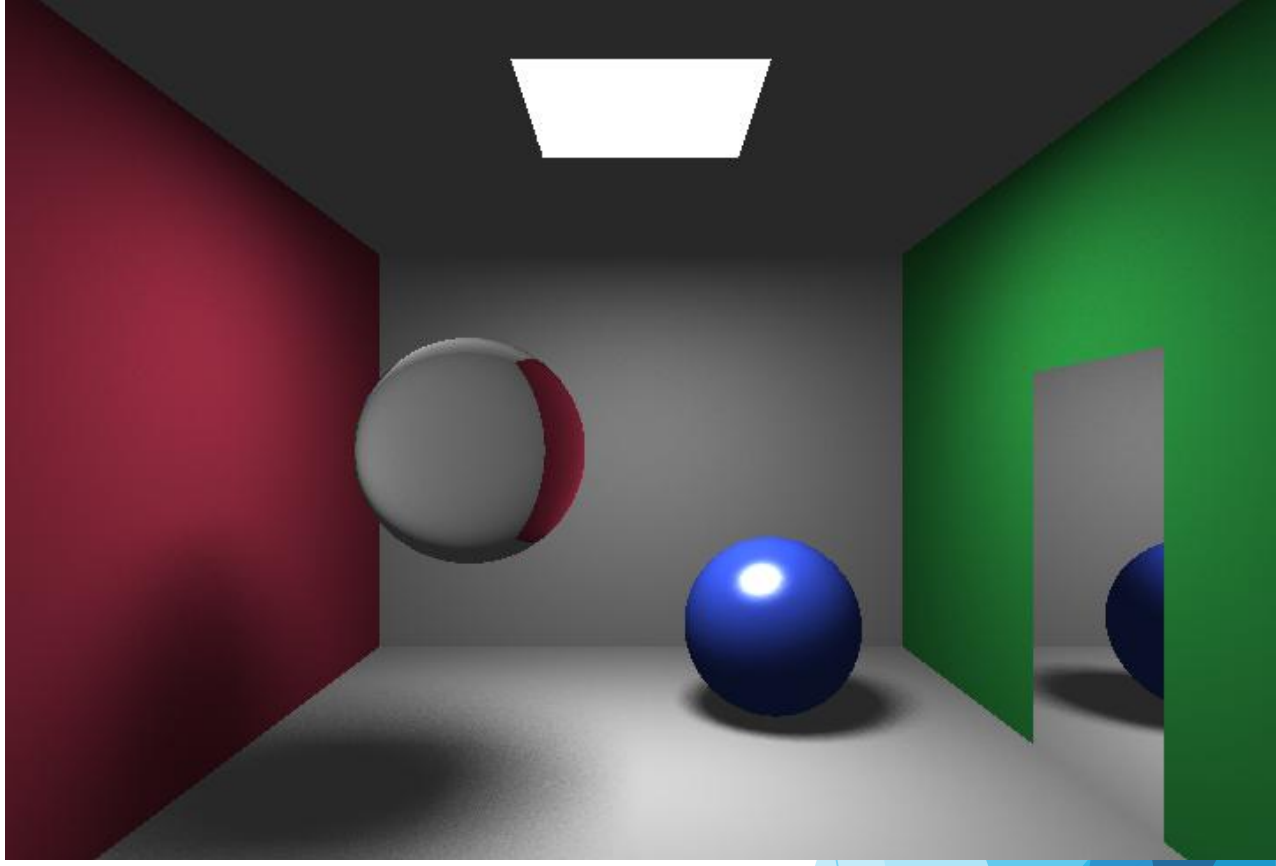
- Using the hemispherical formulation of the Rendering Equation

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega_{2\pi}} L_i(x, \omega_i)\, f_r(\omega_o, \omega_i)\, (N \cdot \omega_i)\, \mathrm{d}\omega_i$$

- Using the area formulation of the Rendering Equation

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{A} L_o(y, -\omega_i)\, f_r(\omega_o, \omega_i)\, G(x, y)\, V(x, y)\, dA_y$$





Last Class Example Results

Left: Solid Angle Sampling (256 samples)
Right: Area Sampling (256 samples)

Direct Illumination

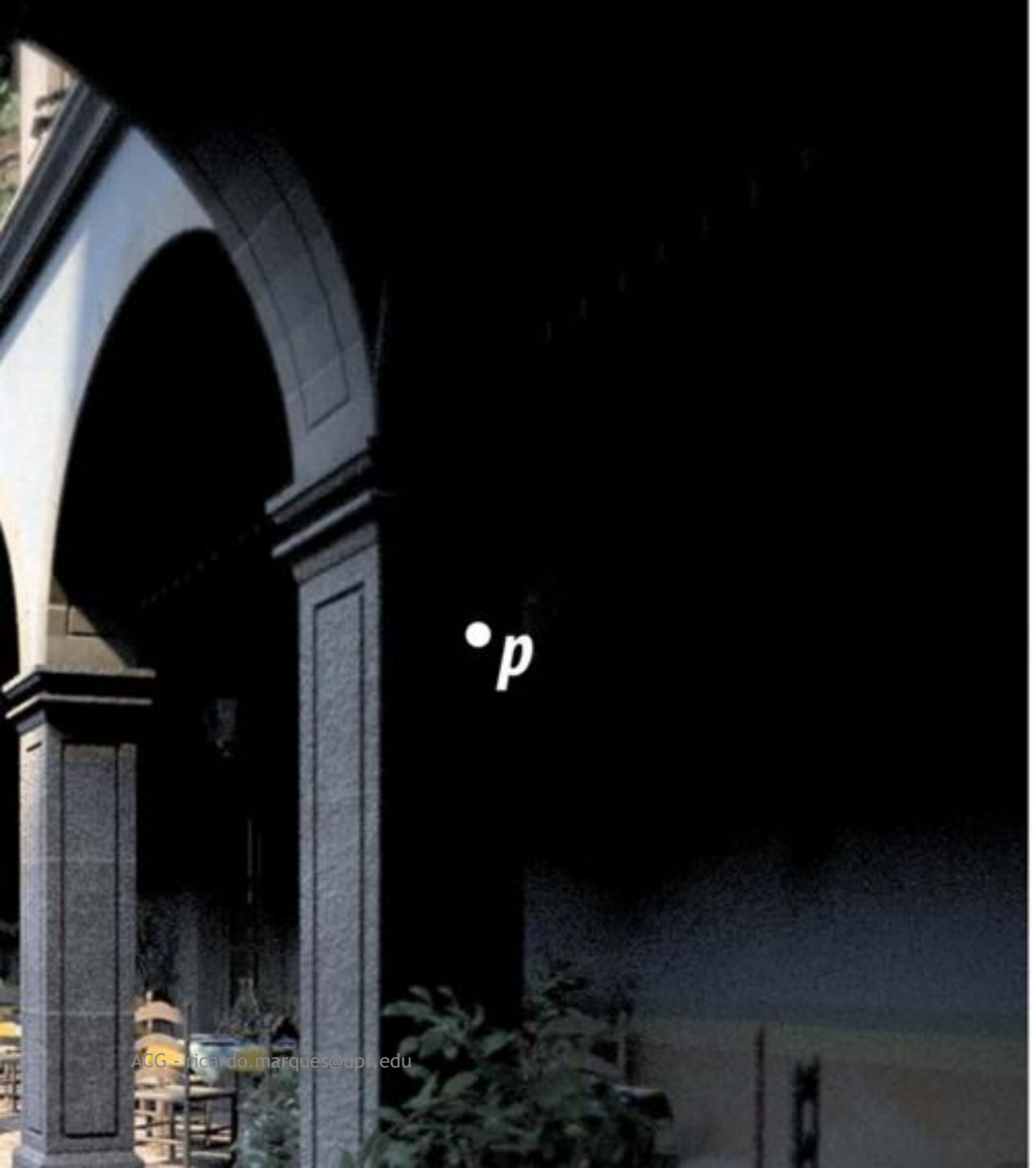
- Only a part of the whole
  - Paths with 0 bounces (light that arrives to the pixel directly from the light sources)
  - Paths with 1 bounce (light that arrives to the pixel after one bounce in the scene)
- Small subset of all possible light paths
  - With direct illumination, paths with >1 bounces are not sampled
  - Important part of the illumination is excluded from the final image
- Let’s see an example



Image from **CMU 15-462/662**



Image from **CMU 15-462/662**


p


p

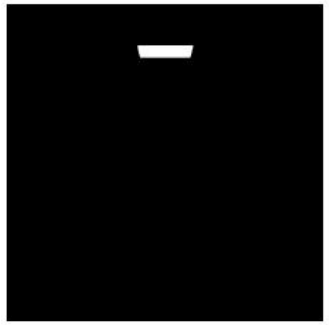
Indirect Illumination



- Indirect Illumination
  - Light that arrives at a given scene point after bouncing more than once through the scene
  - Implies simulating light paths with three or more segments

- Features
  - Varies across the scene (as opposed to the constant approximation used in the Phong Illumination model)
  - Smooth variations (as opposed to the sharp variations of the direct illumination)
  - Can arrive from any direction (as opposed to the direct illumination which can only arrive from the light sources)

0 bounces only



1 bounce only



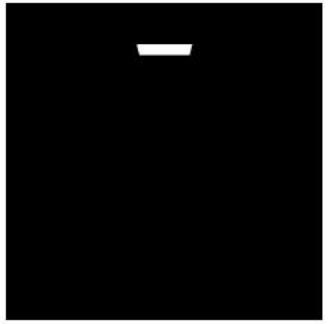
2 bounces only



3 bounces only



0 bounces only



0+1 bounces



0+1+2 bounces

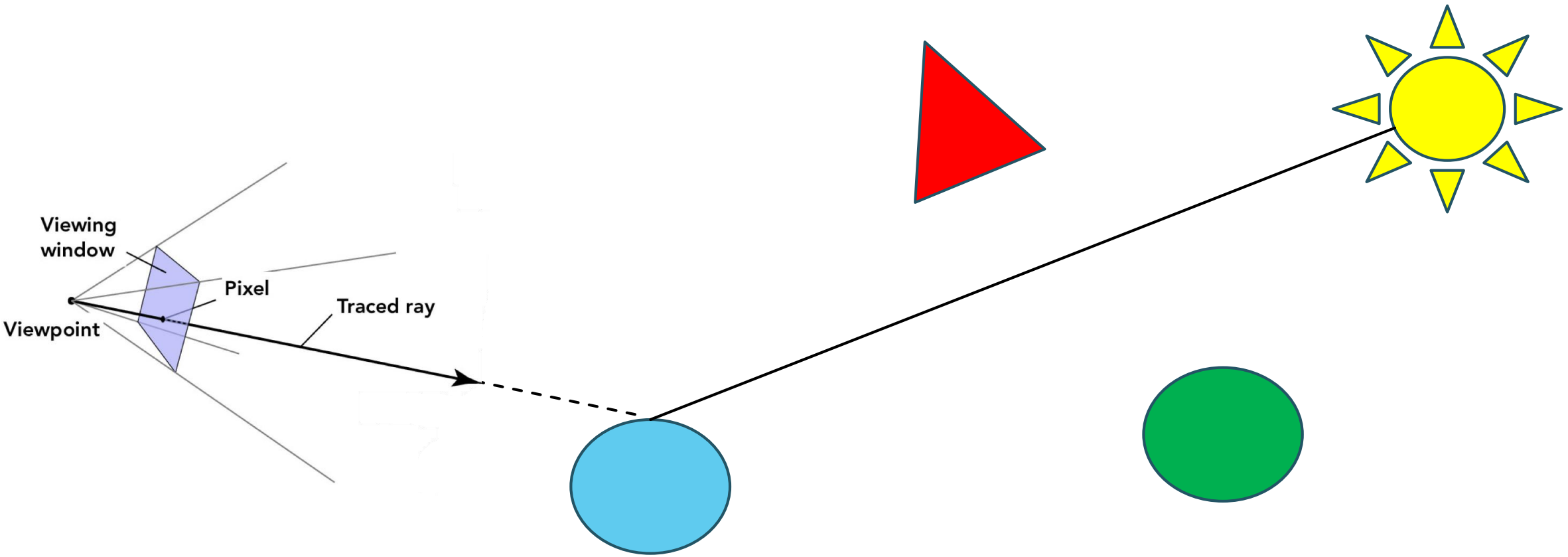


0+1+2+3 bounces

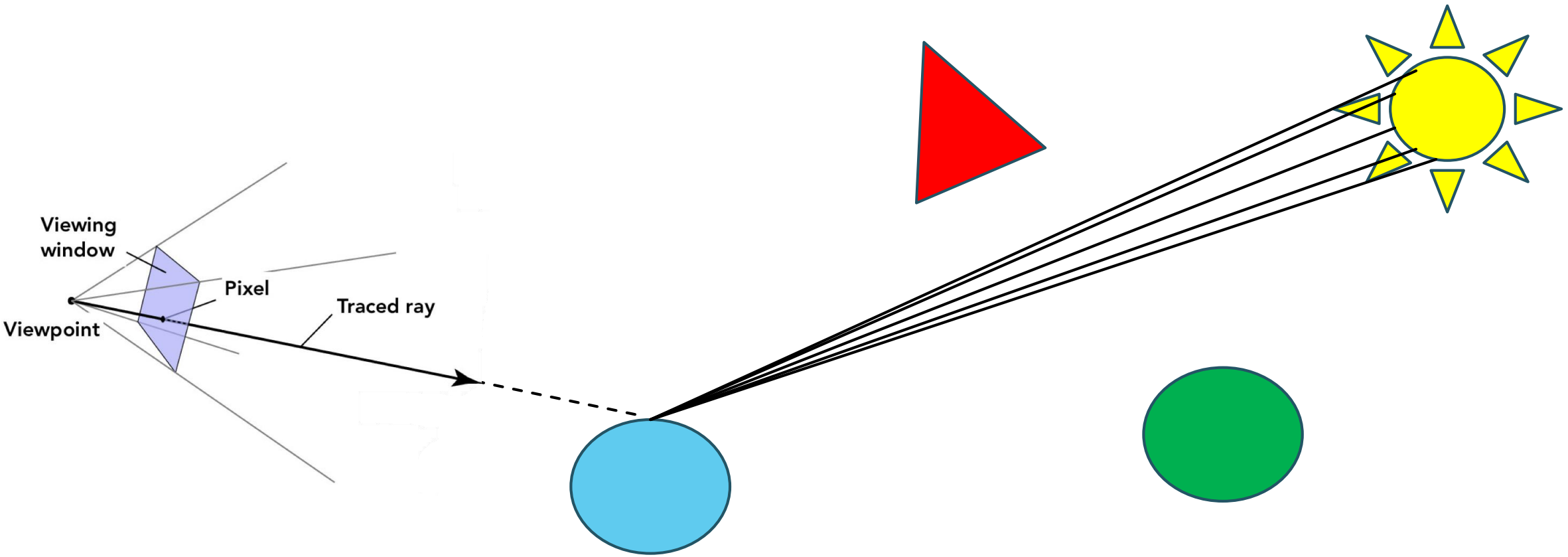


Simulating Longer Paths

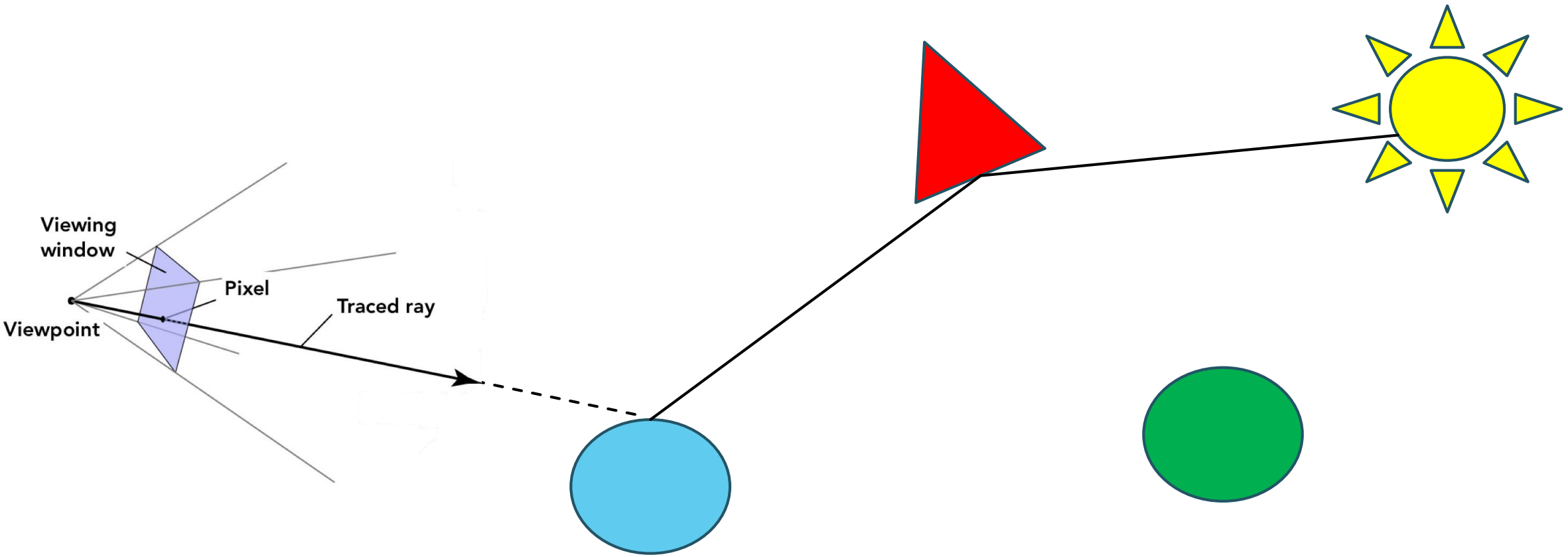
1-Bounce Path Connecting to Light



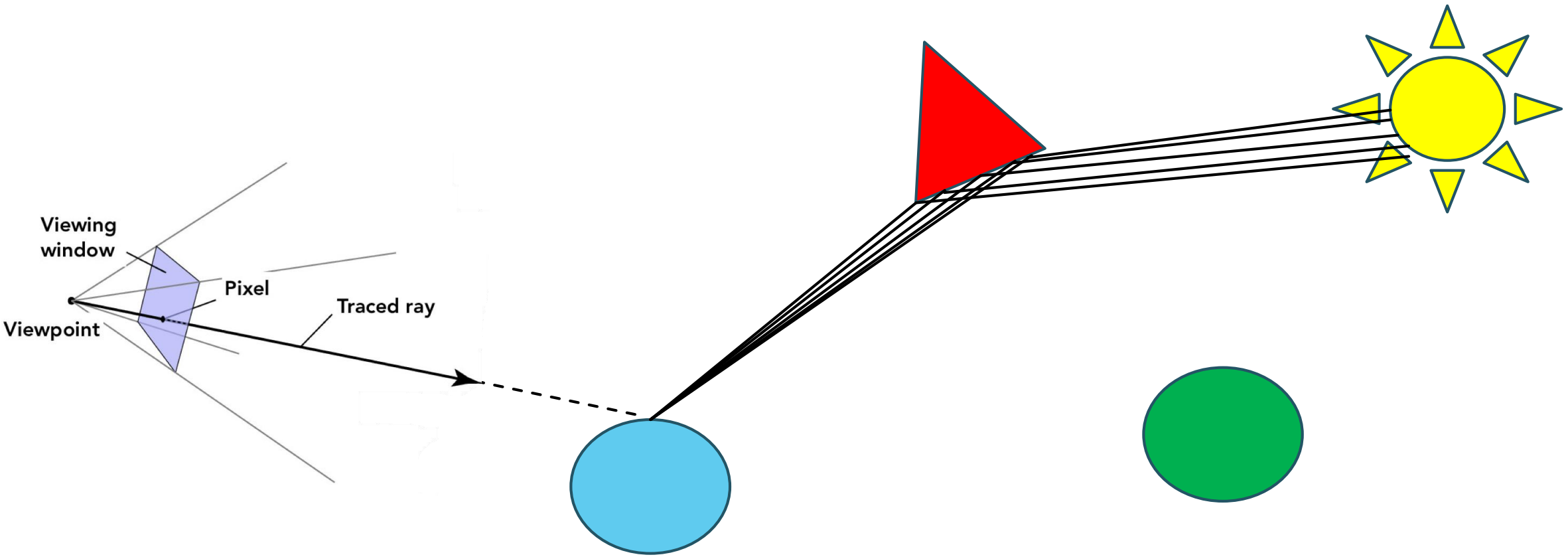
1-Bounce Path Connecting to Light



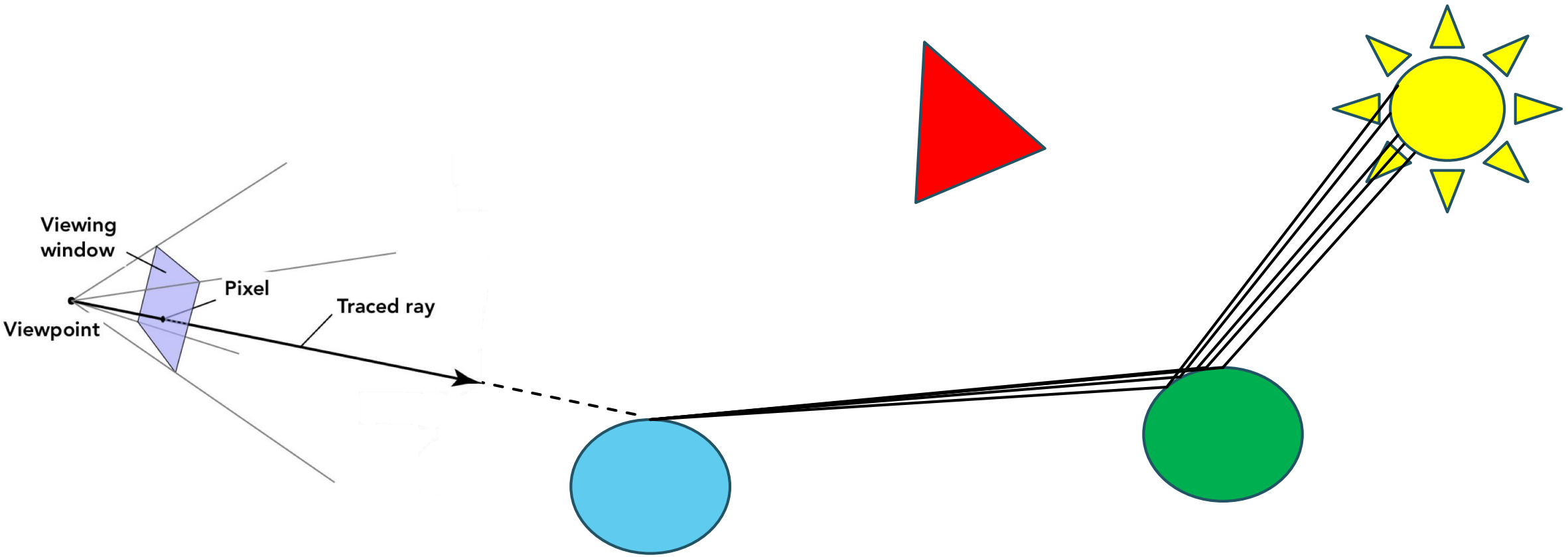
2-Bounce Path Connecting to Light



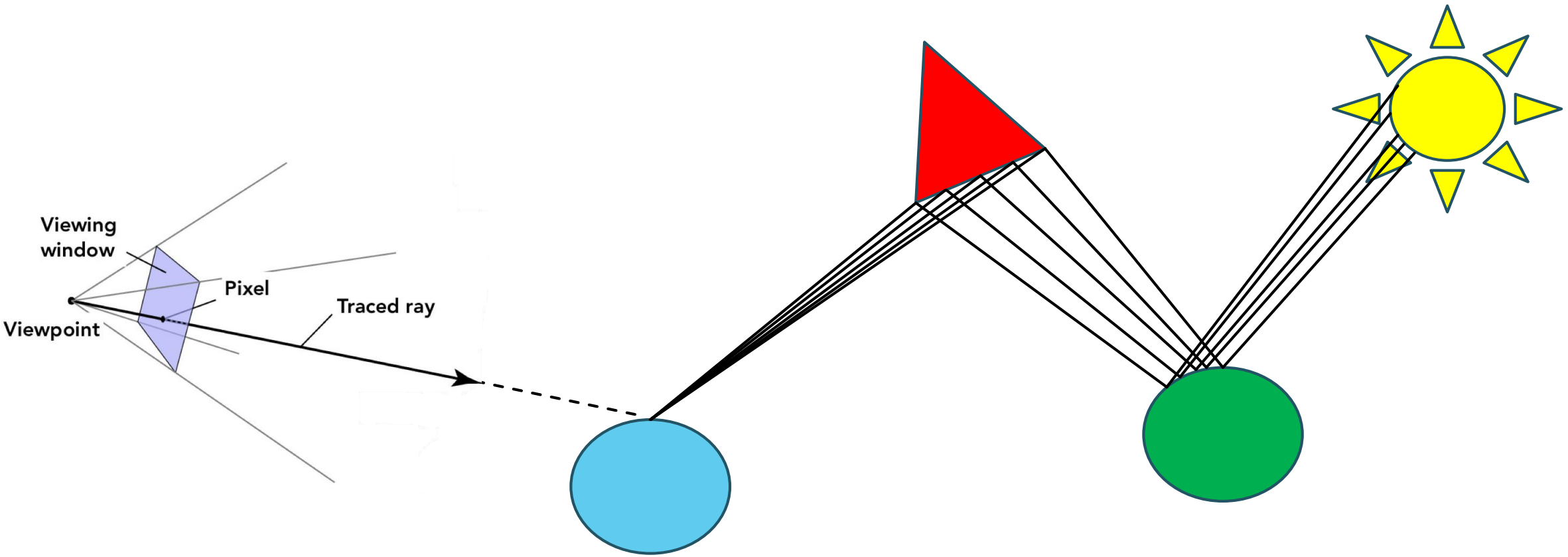
2-Bounce Paths Connecting to Light



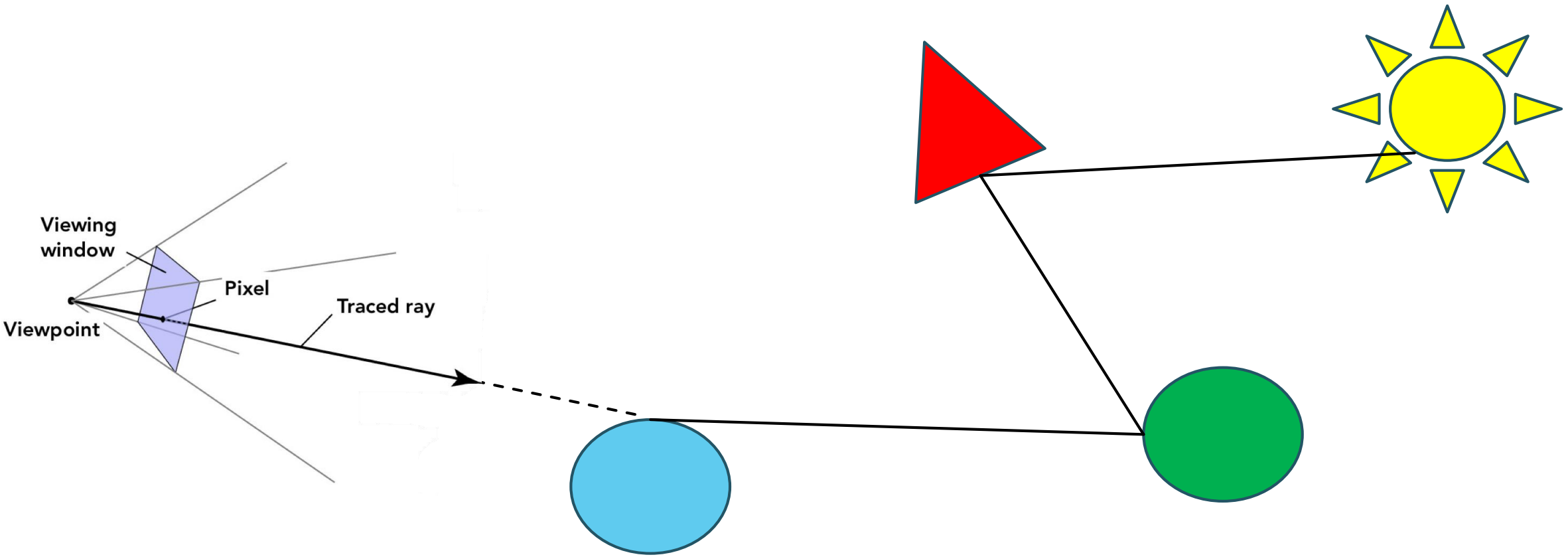
2-Bounce Paths Connecting to Light



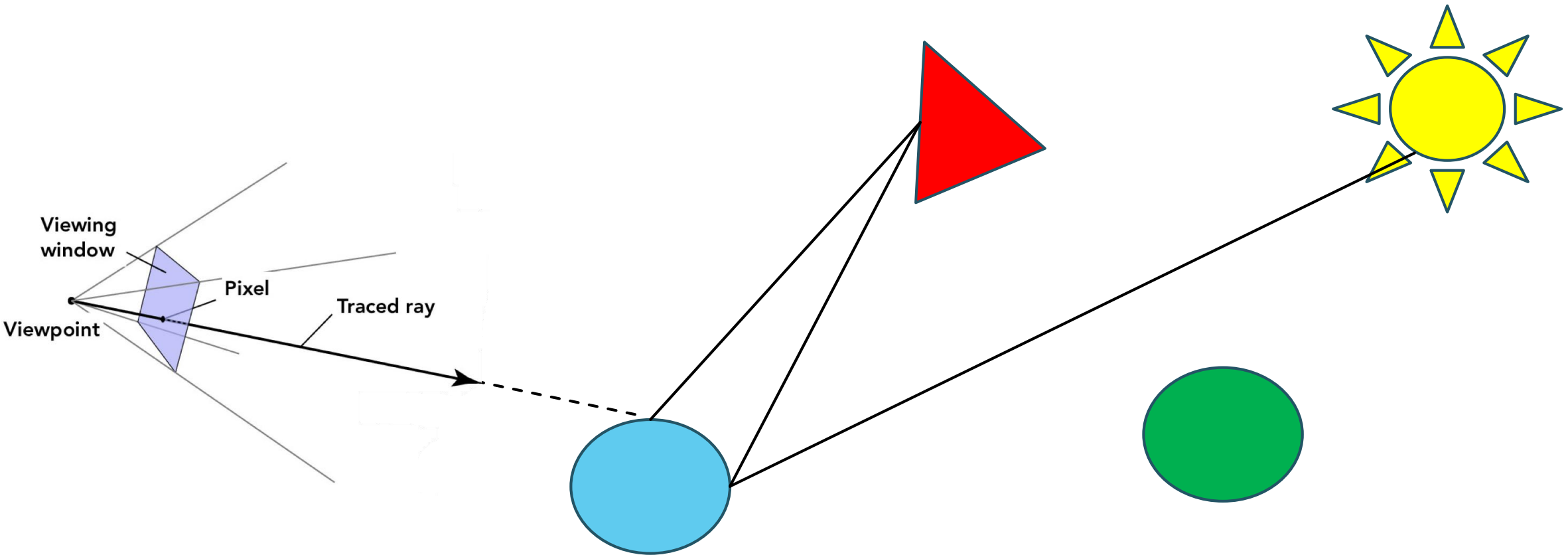
3-Bounce Paths Connecting to Light



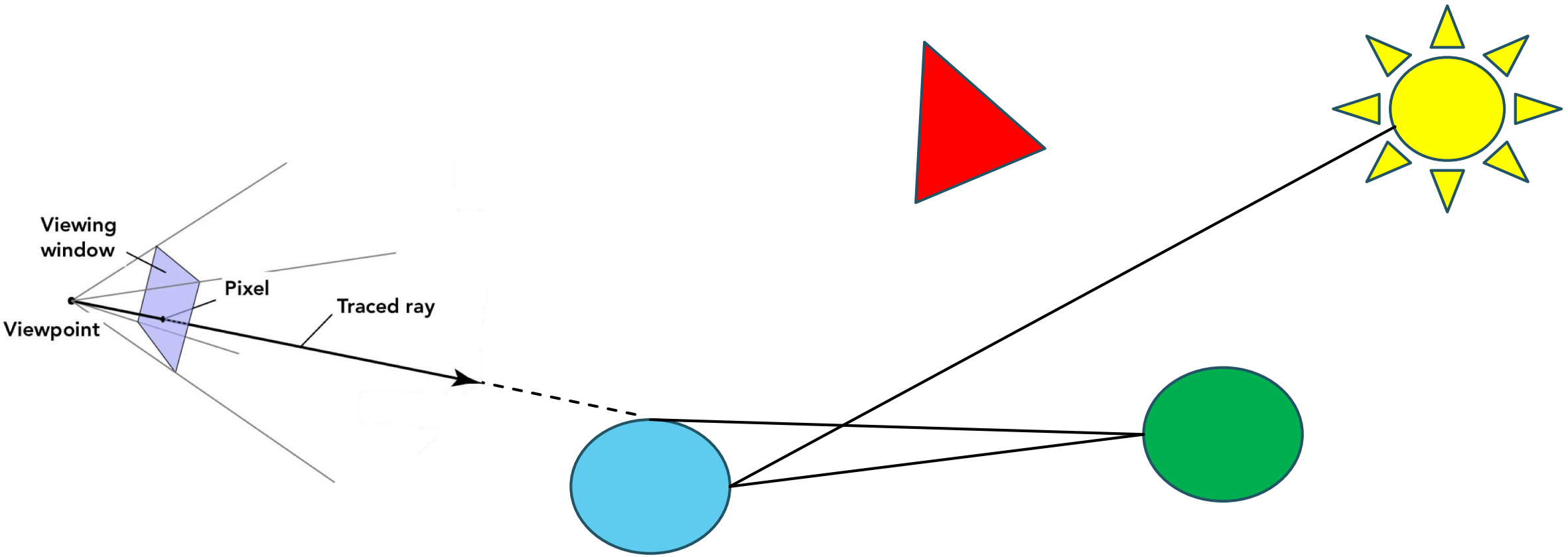
3-Bounce Paths Connecting to Light



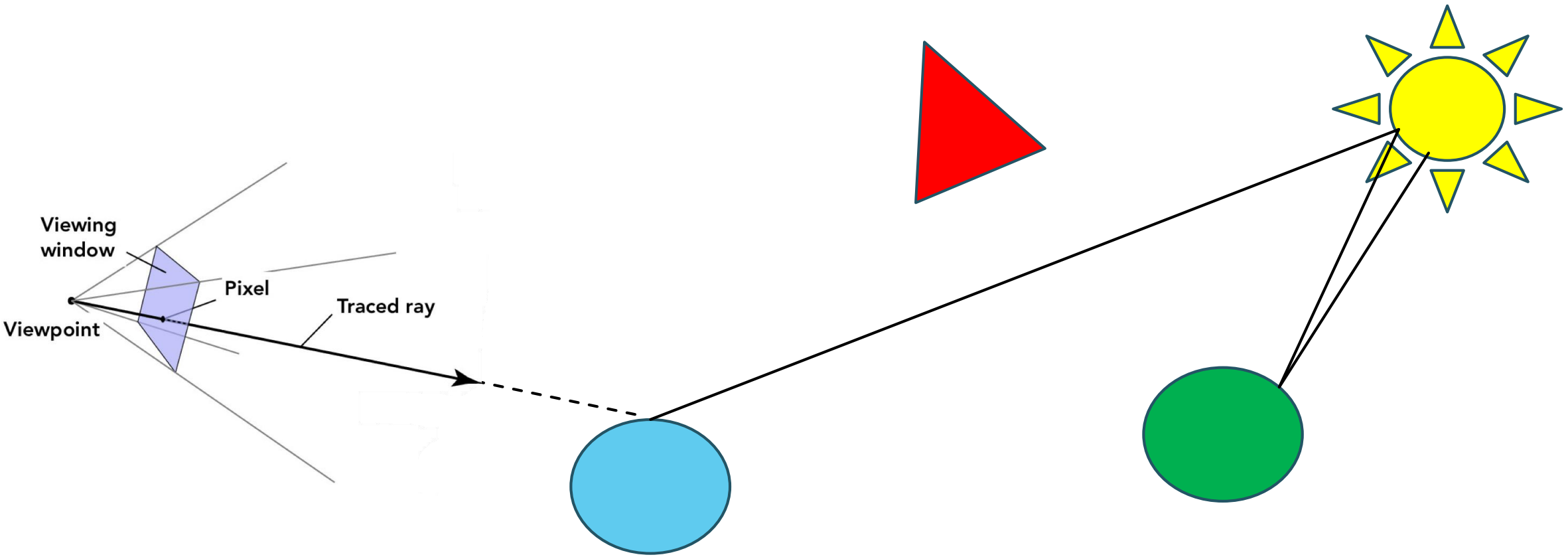
3-Bounce Paths Connecting to Light



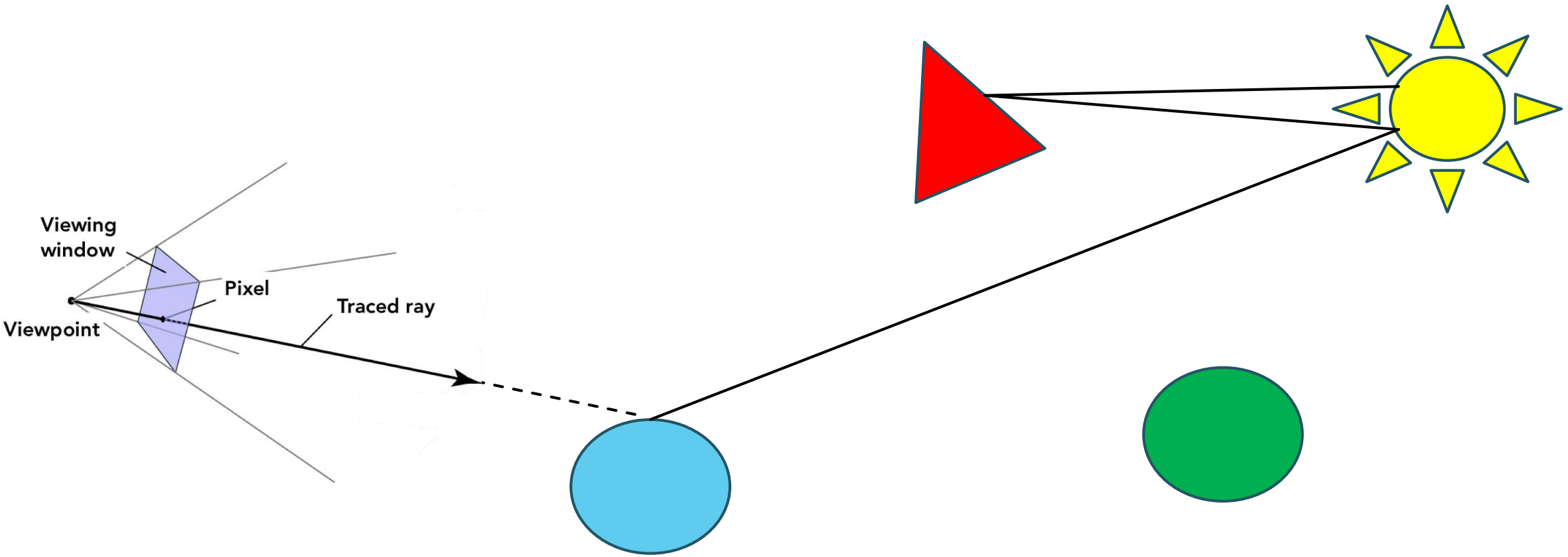
3-Bounce Paths Connecting to Light



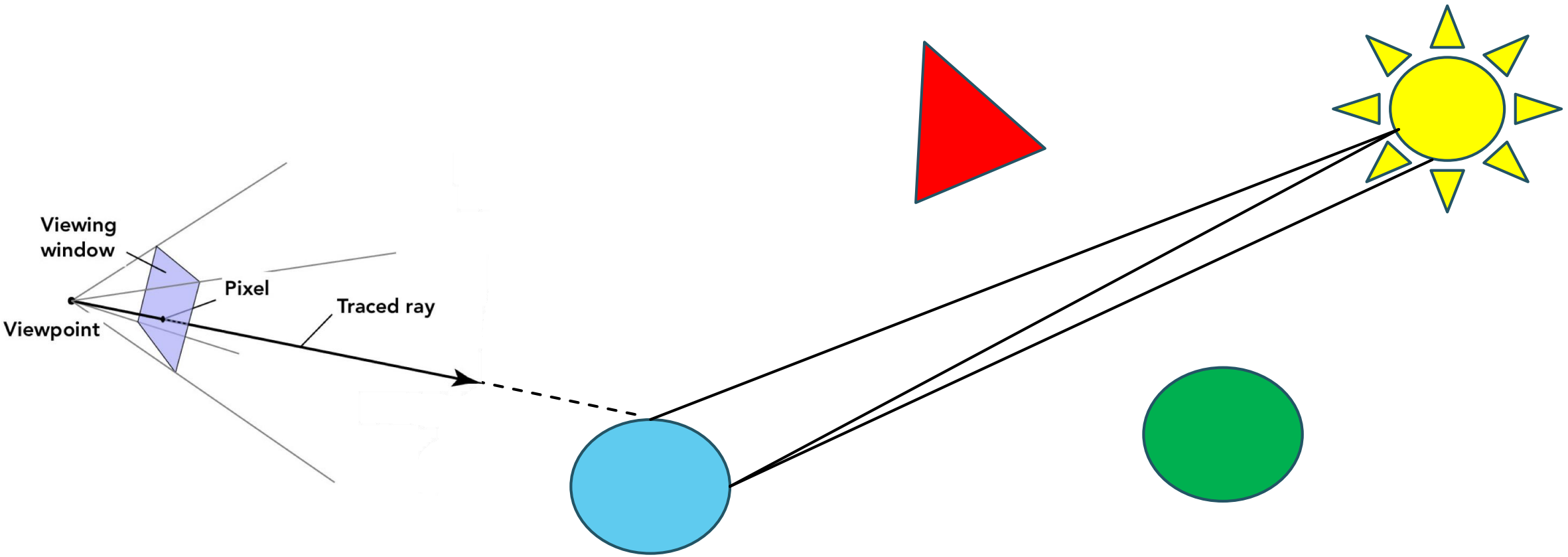
3-Bounce Paths Connecting to Light



3-Bounce Paths Connecting to Light



3-Bounce Paths Connecting to Light

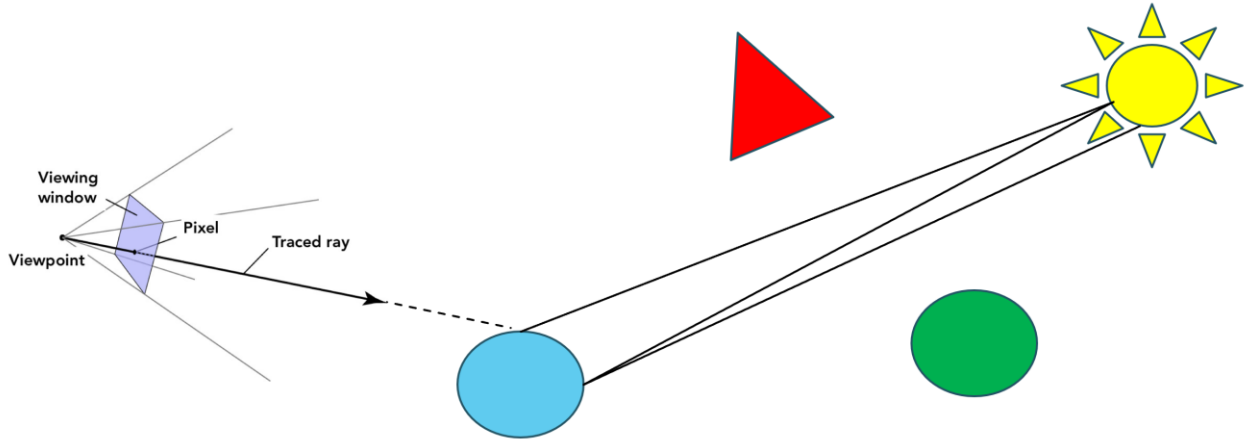


Brainstorming: Global Illumination

- As previously illustrated Global Illumination is about simulating possible light paths on a virtual scene
  - There is an infinite number of possible light paths

- Questions
  - Can we generate all possible paths?
  - What strategy should we take?
  - How to efficiently generate paths?

Simple Path Tracer (V0)



ComputeRadiance(Ray r)

→ L_o = r.firstIntersection().getEmittedLight

→ wi, pdf = Hemispherical sampling

→ if (r.depth < max_depth){

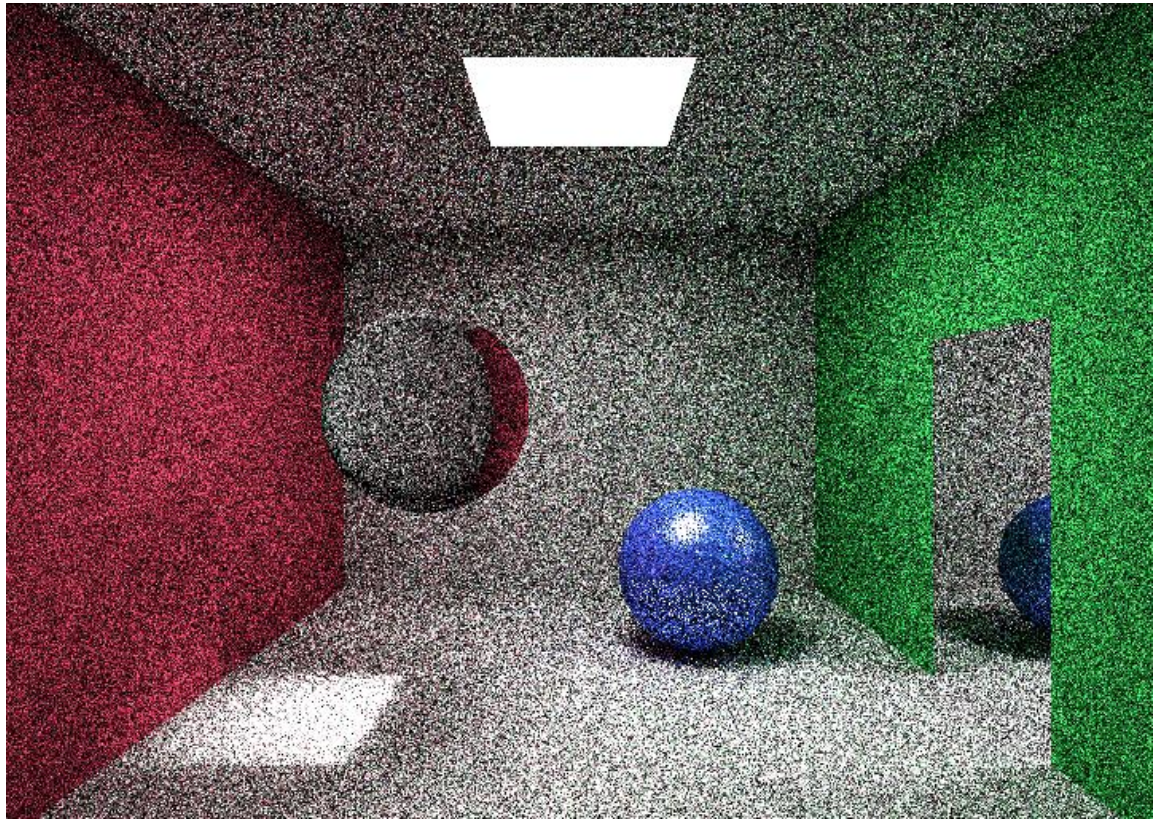
→ → ray_new = Ray(wi, itsPoint)

→ → L_o += ComputeRadiance(ray_new) · BRDF · (wi · N) / pdf

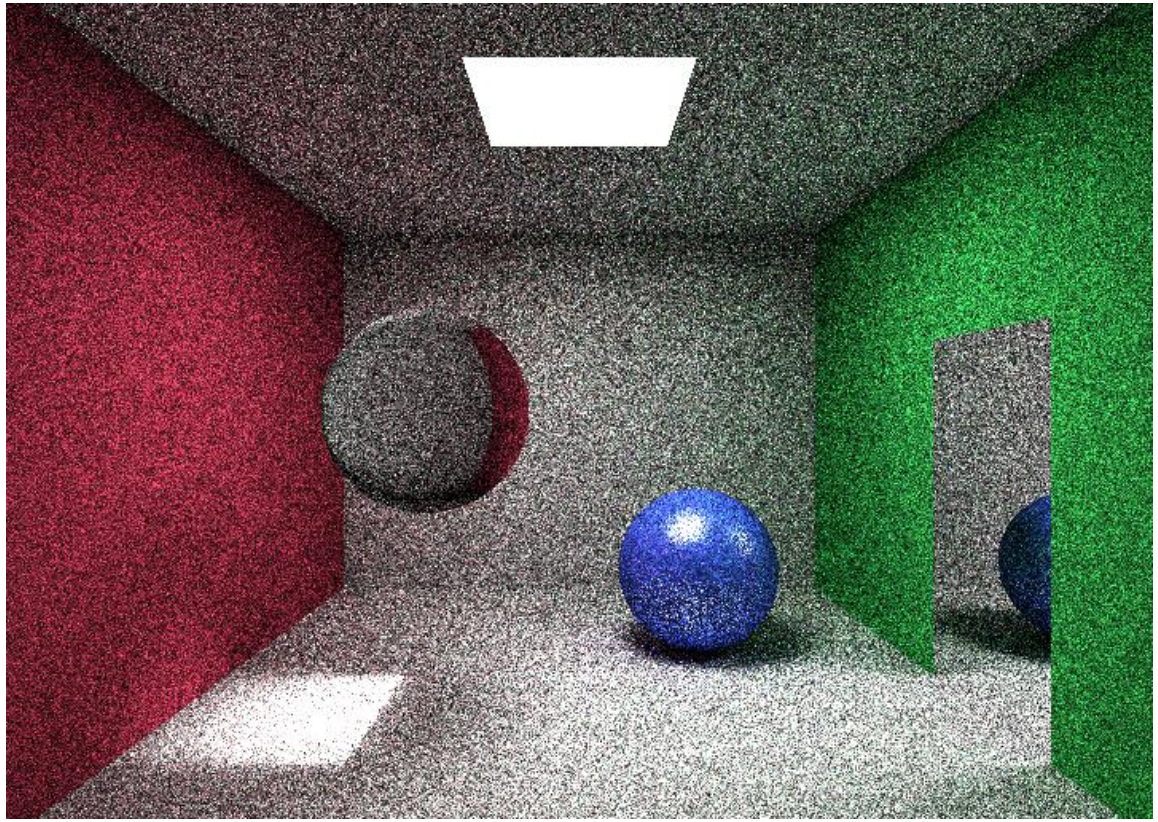
return L_o;

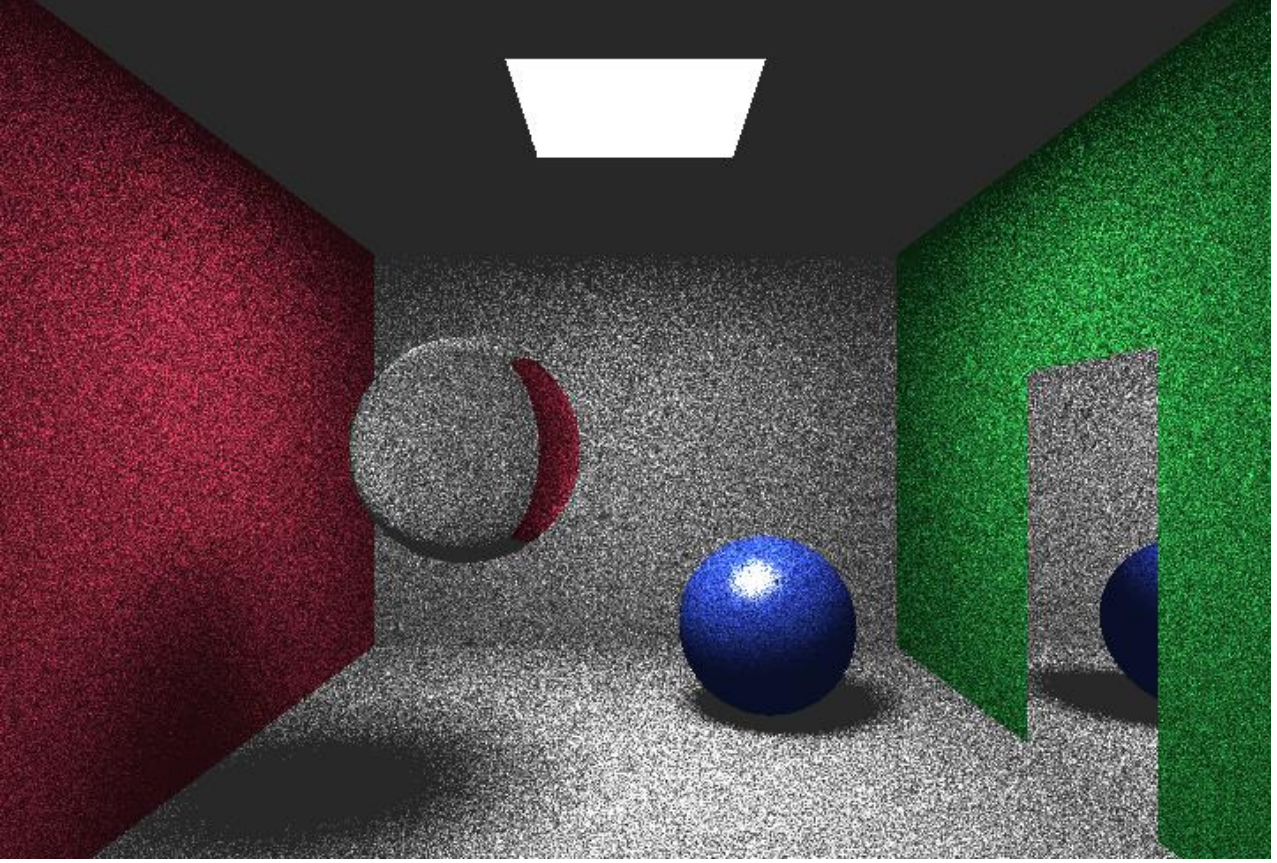
for (int s = 0; s < spp; s++){
→ L_o += ComputeRadiance()
L_o /= spp

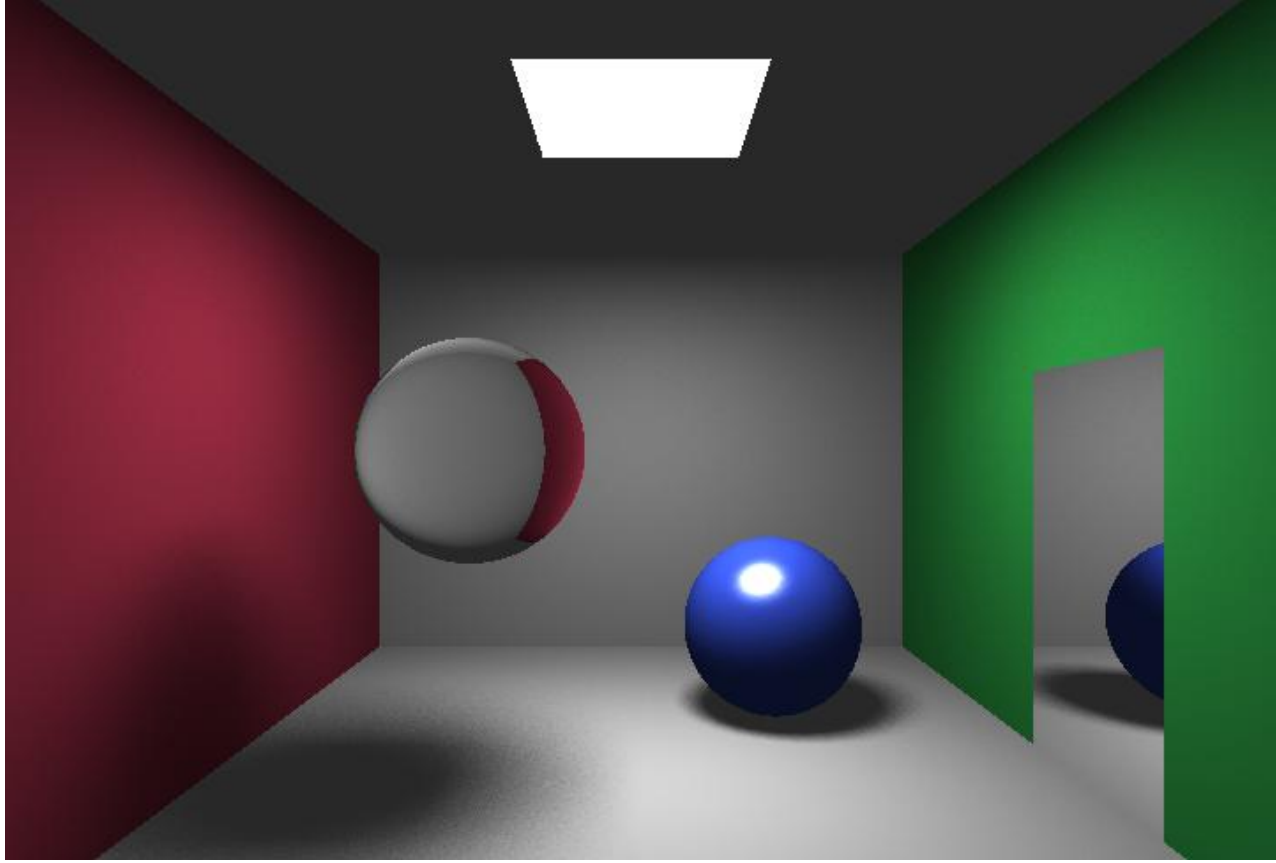
Result Simple Path Tracer
(128 spp, 4 bounces)



Result Simple Path Tracer
(256 spp, 4 bounces)



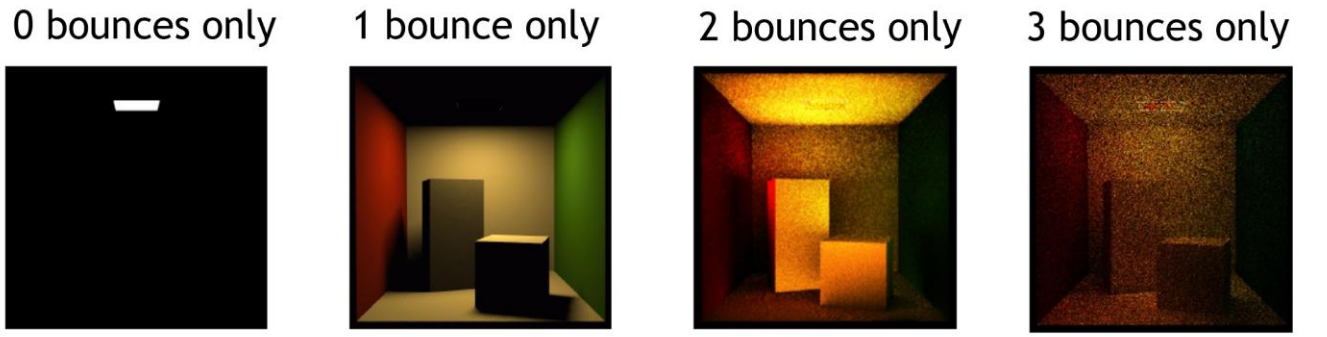


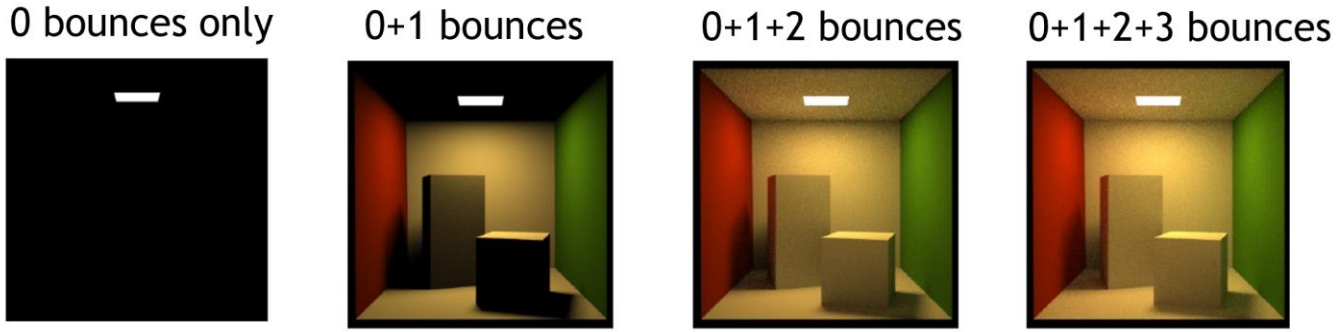


Recall: the importance of choosing the sample directions

Left: Solid Angle Sampling (256 samples)
Right: Area Sampling (256 samples)

How can we improve our simple path tracer?

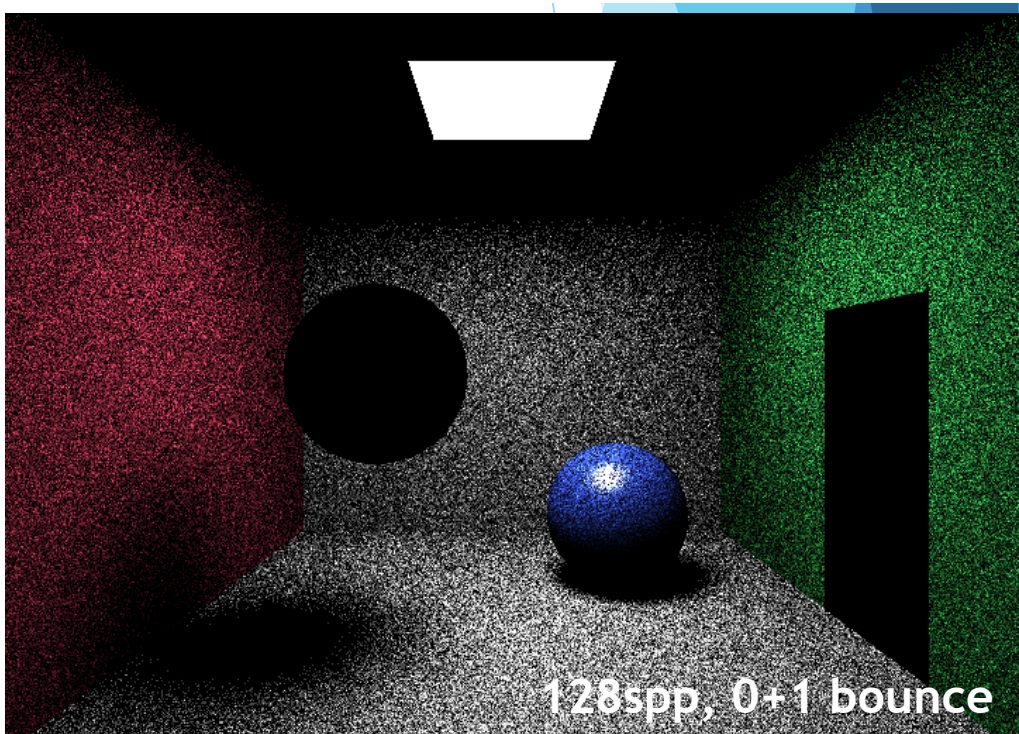




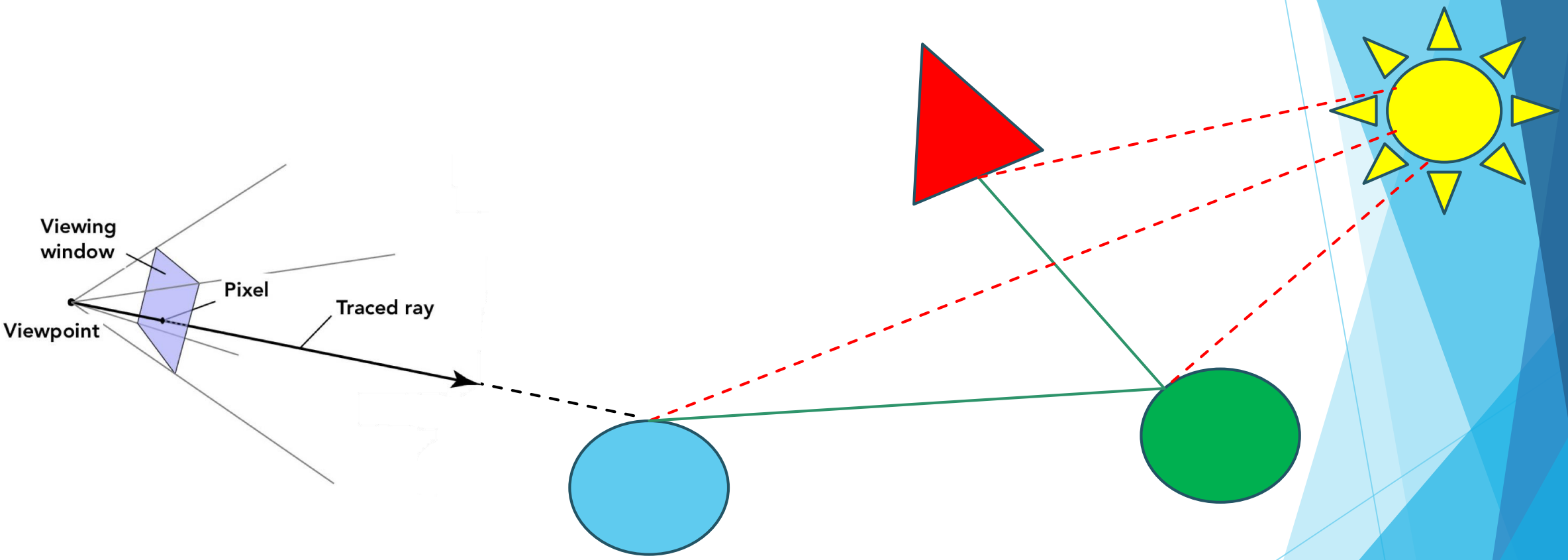
- Limitations
  - Slow
  - Requires (too) many samples for a noise-free image

- Suggestions for improvem
  - Separate the in diance into direct and indirect
  - Comput components separately
  - Sample light sources area for the direct component (more efficient)

Pause

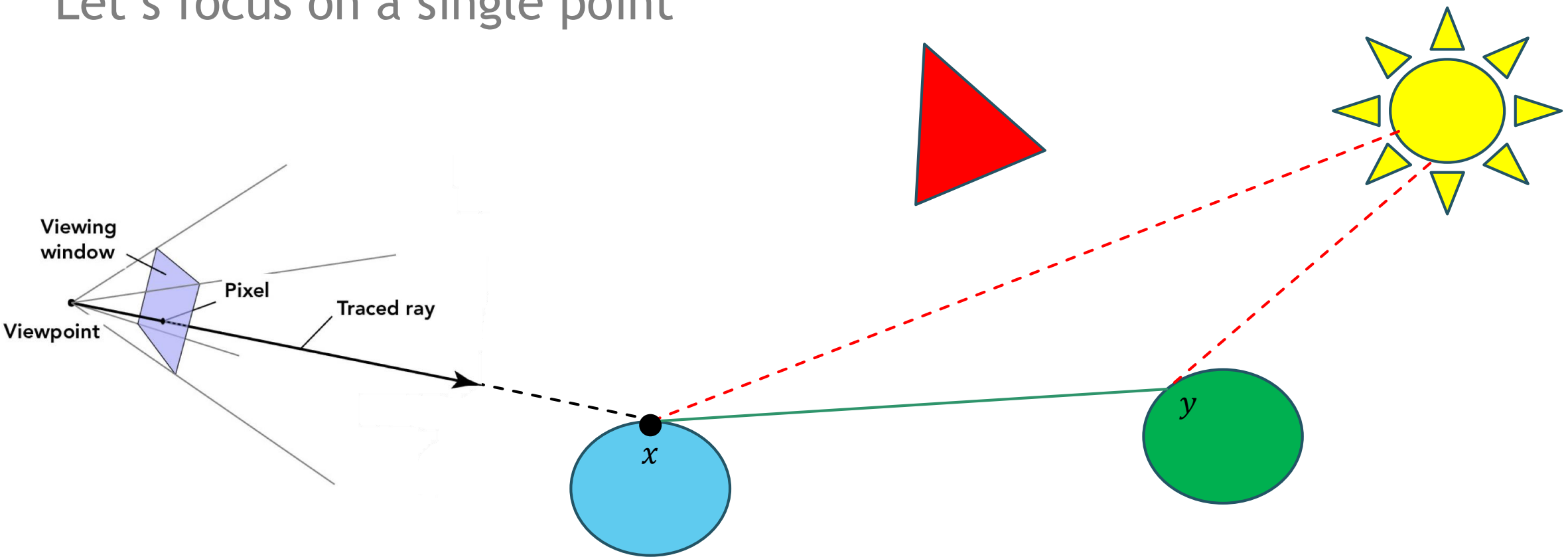


Path Tracing with Next Event Estimation



Towards PT with NEE

Let's focus on a single point



$$L_i(pixel, r.d) = L_o(x, -r.d) = L_e(x, -r.d) + L_r(x, -r.d)$$

$$L_r(x, -r.d) = L_r^{dir}(x, -r.d) + L_r^{ind}(x, -r.d)$$

Path Tracer with NEE

```
ComputeRadiance(ray, depth)
    x     = IntersectScene(ray);
    Lo    = EmittedRadiance(x, -ray.dir) + ReflectedRadiance(x, -ray.dir, depth);
    return Lo;

EmittedRadiance(x, ωo)
    Le = … x.getEmissiveRadiance()
    return  Le;

ReflectedRadiance(x, ωo, depth)
    Lr = … DirectRadiance(x, wo) + IndirectRadiance(x, wo, depth)
    return Lr;
```

only checked here

Path Tracer with NEE

DirectRadiance(x, ωo)

L_dir = ... Vector 3(0) → Emission already counted in EmittedRadiance

$y = SampleLight$, $G = \frac{(N \times w_i) \cdot (-w_i \cdot N_y)}{\|y - x\|^2}$

$w_i = (y - x) / \|y - x\|$

$L_{dir} += BRDF.V(x, w_i) \cdot G / pdf$

return L_dir;

IndirectRadiance(x, ωo, depth)

L_ind = 0

w_i, pdf = randomDirection(x)

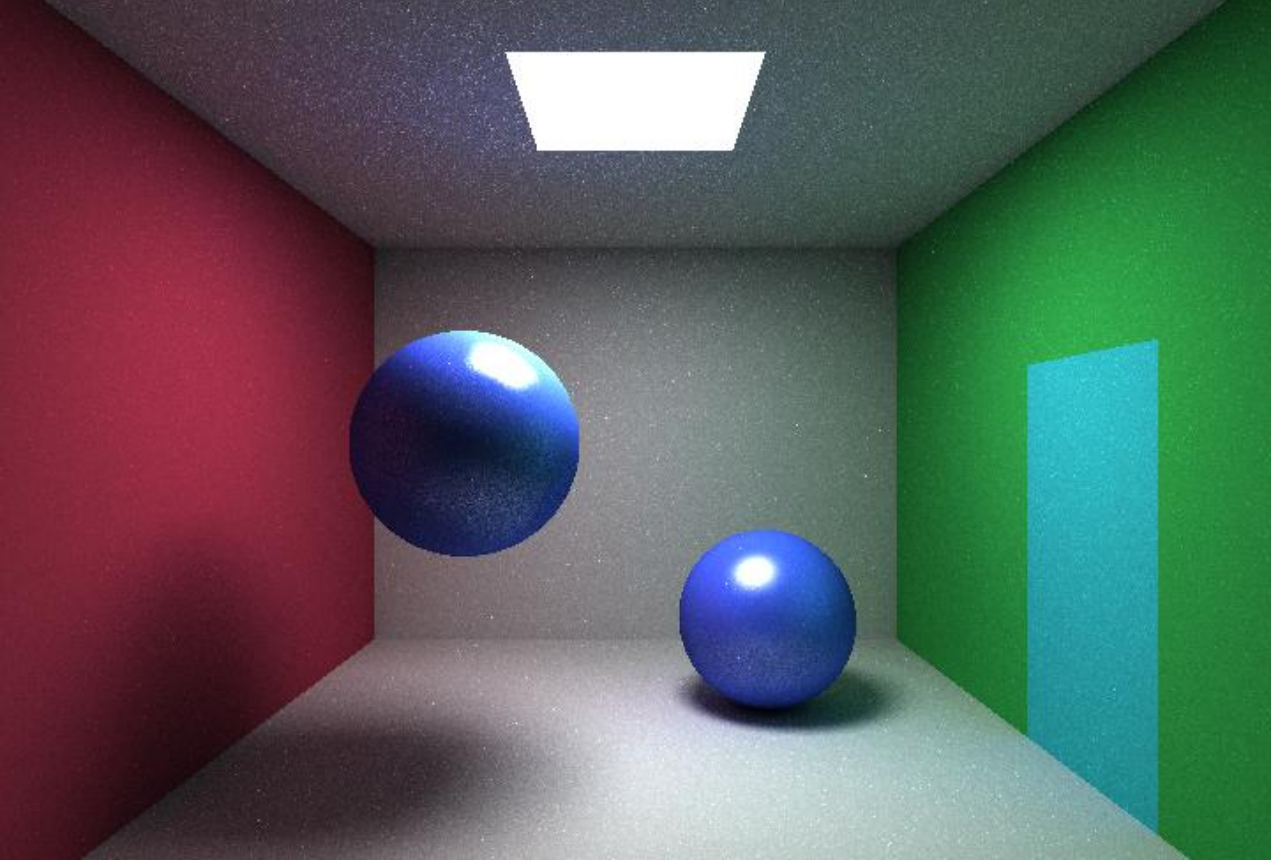
ray_new = Ray(x, w_i)

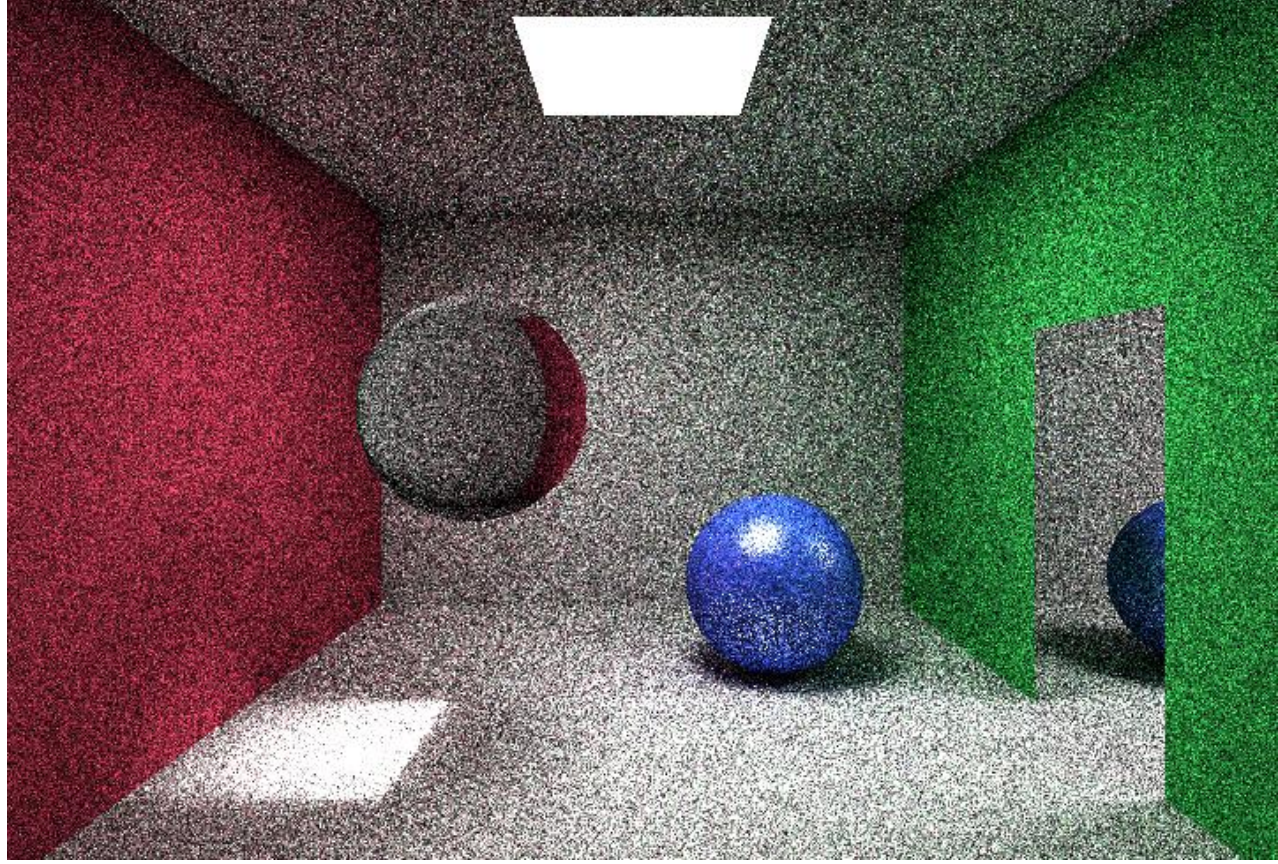
IF depth ^ max_depth { L_ind = ReflectedRadiance(x, w_o, depth) · BRDF · (w_i · N) / pdf

return L_ind;

Emission not counted twice since we do ReflectedRadiance, which never calls getEmissionRadiance

After reaching max_depth, last ReflectedRadiance will return 0 but L_ind will already have accumulated results





Left: PT with NEE (256 spp)
Right: Simple PT (256 spp)

Advanced Path Tracing

Importance Sampling

Expected Value of MC Estimates

- Consider the integral I and the Monte Carlo estimator $\hat{I}$ such that:

$$I = \int_a^b f(x)\,\mathrm{d}x$$

$$\hat{I} = \frac{1}{N}\sum_{i=1}^{N}\frac{f(x_i)}{p(x_i)}$$

- Using the properties of expected values, we can show that $E[\hat{I}] = I$

Expected Value - Proof

$$\mathrm{E}\left[\hat{I}\right] = \mathrm{E}\left[\frac{1}{N}\sum_{j=1}^{N}\frac{f(x_j)}{p(x_j)}\right]$$

$$= \frac{1}{N}\,\mathrm{E}\left[\sum_{j=1}^{N}\frac{f(x_j)}{p(x_j)}\right]$$

$$= \frac{1}{N}N\,\mathrm{E}\left[\frac{f(x)}{p(x)}\right]$$

$$\stackrel{\text{def}}{=} \int_a^b \frac{f(x)}{p(x)}p(x)\,\mathrm{d}x = I$$

$\hat{I}$ is an **unbiased** estimator of I

- No systematic error
- But there is (random) error... ← Variance

For biased estimators, we have:

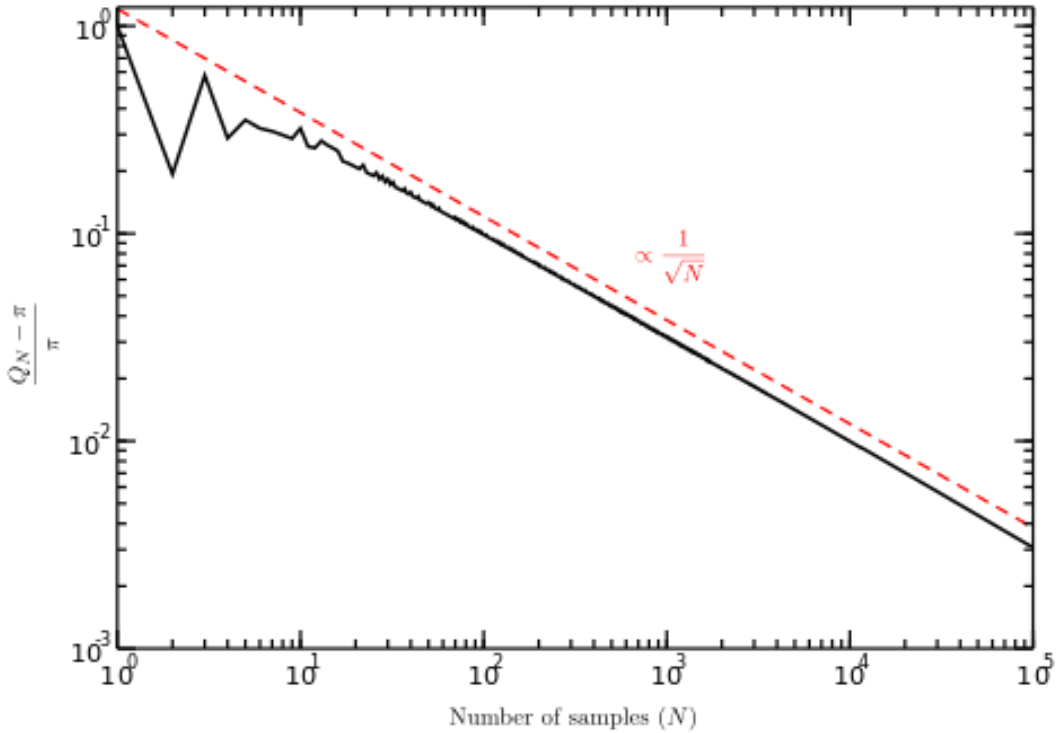
$$E\left[\hat{I}\right] = I + B,$$

where B is the bias

Variance of MC Estimates

- Using the properties of variance, we can show that:

$$V[\hat{I}] = \frac{1}{N}\left(\int_a^b \frac{f(x)^2}{p(x)}\,\mathrm{d}x - I^2\right)$$



- The standard deviation of the variance $\sigma = \sqrt{V[\hat{I}]} \propto 1/\sqrt{N}$
  - Very slow convergence
  - Need to multiply N by 4 to reduce σ by half (!)

Variance (proof)

$$
\begin{aligned}
V[\hat{I}] &= V\left[\frac{1}{N}\sum_{j=1}^{N}\frac{f(x_j)}{p(x_j)}\right] \\
&= \frac{1}{N^2}\sum_{j=1}^{N} V\left[\frac{f(x_j)}{p(x_j)}\right] \\
&= \frac{1}{N^2} N\, V\left[\frac{f(x)}{p(x)}\right] \\
&= \frac{1}{N}\left(\mathrm{E}\left[\left(\frac{f(x)}{p(x)}\right)^2\right] - \mathrm{E}\left[\frac{f(x)}{p(x)}\right]^2\right) \\
&= \frac{1}{N}\left(\int_a^b \frac{f(x)^2}{p(x)}\,\mathrm{d}x - I^2\right)
\end{aligned}
$$



Some Observations

Monte Carlo methods converge slowly

We would like that it converges faster, that is, to have a smaller error (variance)

But what does the variance of a MC estimator depend on?

Variance Reduction

$$I = \int_a^b f(x)\,\mathrm{d}x$$

$$\hat{I} = \frac{1}{N}\sum_{j=1}^{N}\frac{f(x_j)}{p(x_j)}$$

$$V[\hat{I}] = \frac{1}{N}\left(\int_a^b \frac{f(x)^2}{p(x)}\,\mathrm{d}x - I^2\right)$$

Degrees of freedom:

- N (the number of samples)
- Probability density function $p(x)$

But how should we choose $p(x)$?

- Idea: choose $p(x)$ that minimizes $V[\hat{I}]$

Variance Reduction

$$I = \int_a^b f(x)\,\mathrm{d}x$$

$$\hat{I} = \frac{1}{N}\sum_{j=1}^{N}\frac{f(x_j)}{p(x_j)}$$

$$V[\hat{I}] = \frac{1}{N}\left(\int_a^b \frac{f(x)^2}{p(x)}\,\mathrm{d}x - I^2\right)$$

Taking $p(x) = \frac{f(x)}{I}$ yields zero variance (proof in next slide)

Ideal pdf thus has the same shape as the integrand $f(x)$

However:

- Implies knowing $f(x)$ and I, which are our unknown quantities!
- Not possible in practice

Proof

$$\mathrm{V}[\hat{I}] = \sigma^2 = \frac{1}{N}\left(\int_a^b \frac{f(x)^2}{p(x)}\,\mathrm{d}x - I^2\right)$$

Let's take $p(x) = c\,f(x)$ where c is a constant such that $c = \frac{1}{\int f(x)dx} = \frac{1}{I}$

Then we have:

$$\int_a^b \frac{f(x)^2}{p(x)}\,\mathrm{d}x = \int_a^b \frac{f(x)^2}{c\,f(x)}\,\mathrm{d}x = \int_a^b \frac{f(x)}{c}\,\mathrm{d}x = \frac{1}{c}\int_a^b f(x)\,\mathrm{d}x = I \times I = I^2$$

And thus $\mathrm{V}[\hat{I}] = 0$

Variance Reduction

$$I = \int_a^b f(x)\,\mathrm{d}x$$

$$\hat{I} = \frac{1}{N}\sum_{j=1}^{N}\frac{f(x_j)}{p(x_j)}$$

$$V[\hat{I}] = \frac{1}{N}\left(\int_a^b \frac{f(x)^2}{p(x)}\,\mathrm{d}x - I^2\right)$$

In real application cases we try to “guess” the shape of the integrand based on some prior knowledge

- Example: the BRDF shape for the case of the illumination integral (more details later)
- This technique of choosing a (suitable) non-uniform pdf is called “***Importance Sampling***”

Monte Carlo Importance Sampling

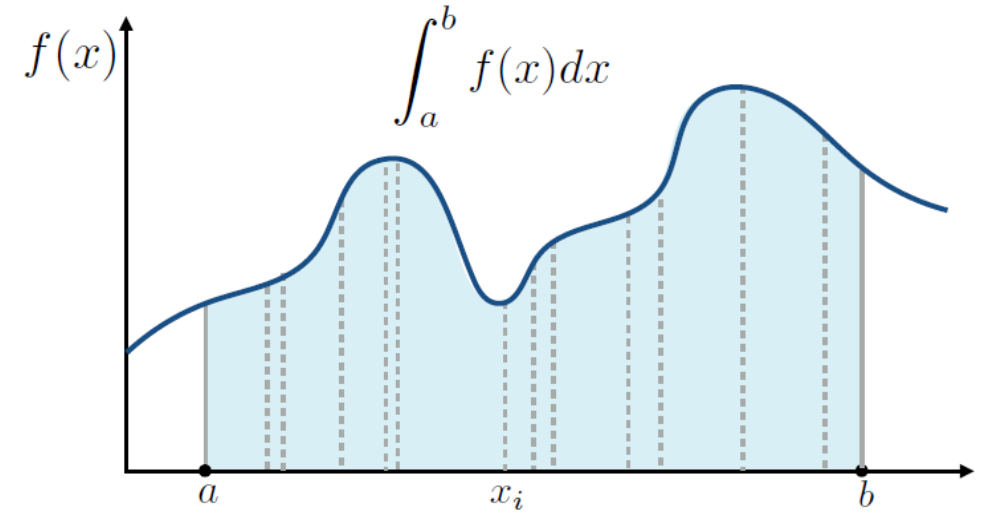
The case of the Illumination Integral

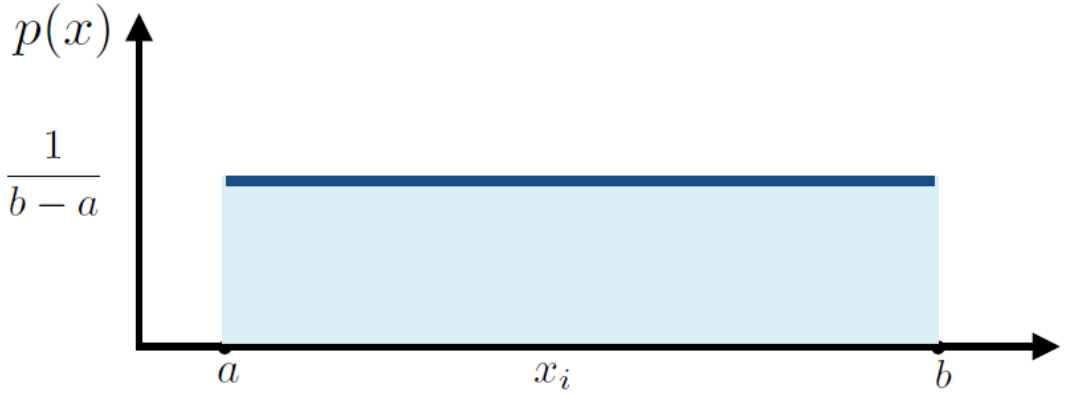
MC with Uniform Sampling

Samples are uniformly distributed over the integration domain

$$p(x) = \frac{1}{b-a}$$

Non-informed way of choosing the sampling strategy

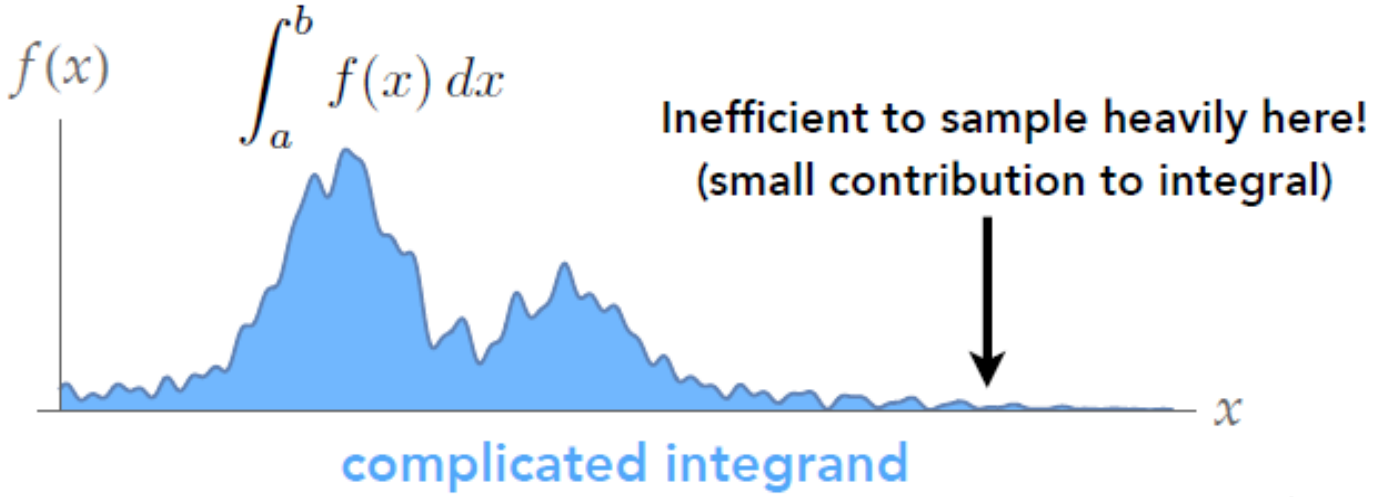




Some more observations

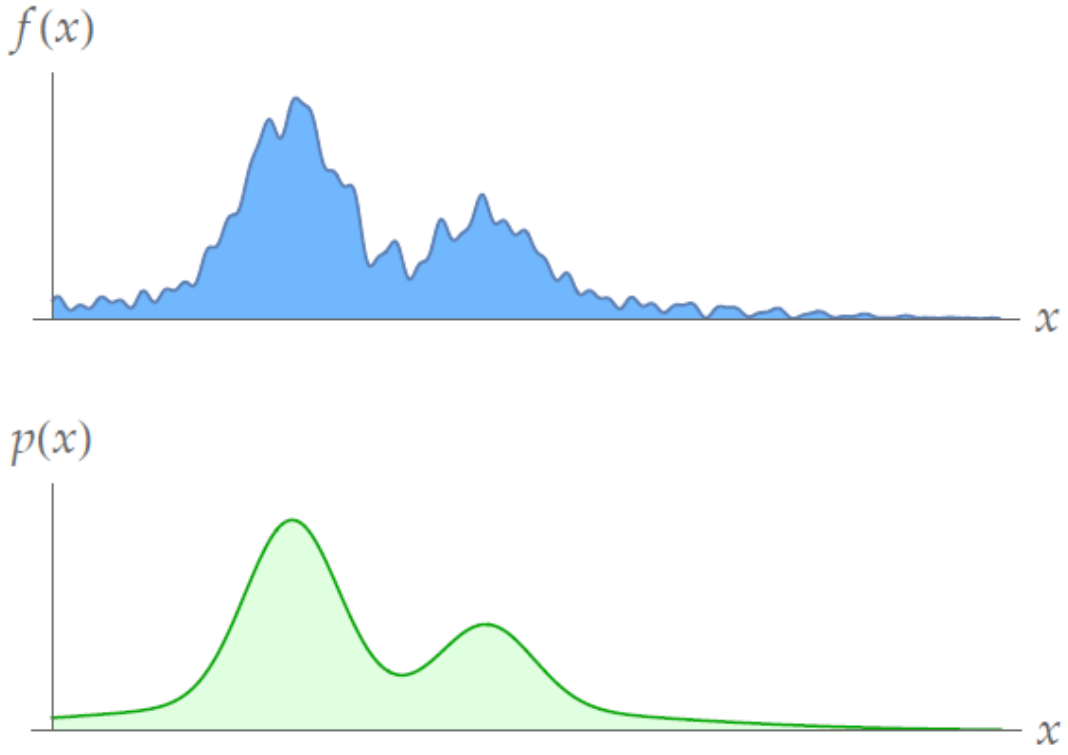
Uniform sampling

- Many samples might be located in non-contributive parts of the integrand
- Poor use of sampling effort (larger convergence time)



Importance Sampling

- Use a pdf $p(x)$ which mimics integrand shape
- Samples are concentrated where $f(x)$ has a larger value
- Better use of sampling effort (faster convergence)



Some more observations

Using importance sampling is not necessarily more efficient

- "Bad" pdf might be worse than uniform sampling
- How to choose the pdf then?

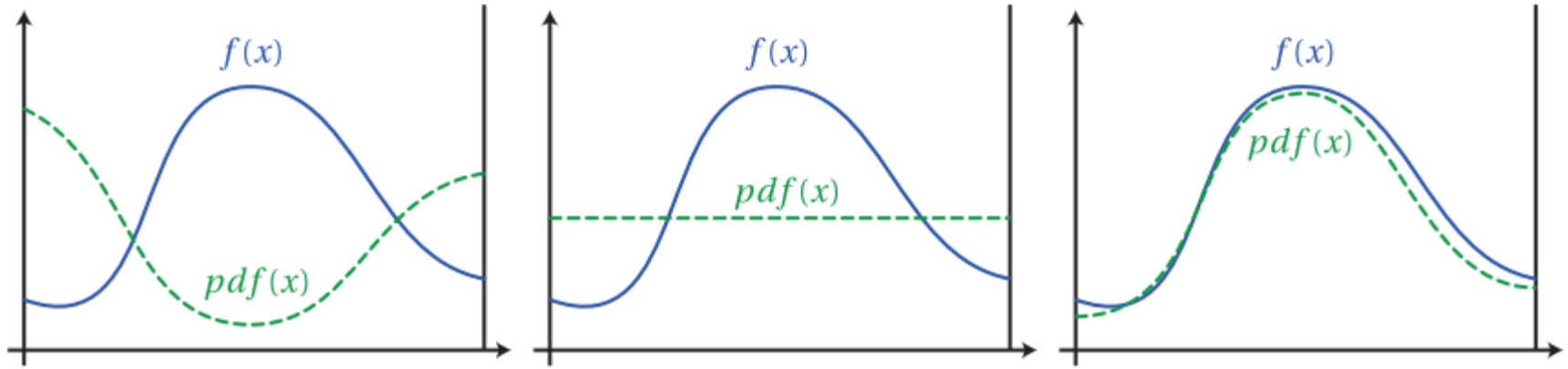


Figure A.2: Comparison of three probability density functions. The PDF on the right provides variance reduction over the uniform PDF in the center. However, using the PDF on the left would significantly increase variance over simple uniform sampling.

Importance Sampling in Practice

Use our prior knowledge about the integrand shape

$$L_r(\omega_o) = \int_{\Omega(x)} L_i(\omega_i) f_r(\omega_i, \omega_o) \cos\theta_i \, \mathrm{d}\omega_i$$

What is known?

- BRDF $f_r(\omega_i, \omega_o)$
- Cosine term

What is unknown?

- The incident radiance function $L_i(\omega_i)$

Importance Sampling in Practice

Case 1 (diffuse BRDF)

$$f_r(\omega_n, \omega_o) = \frac{k_d}{\pi}$$ ← (normalized)

Then, the estimator can be written as:

$$\hat{L}_r(\omega_o) = \frac{k_d}{\pi N} \sum_{n=1}^{N} \frac{L_i(\omega_n) \cos\theta_n}{p(\omega_n)}$$ (just taking k_d/π out of MT)

And we typically make cosine-based importance sampling taking:

$$p(\omega) = \frac{\cos\theta_n}{\pi} \rightarrow \hat{L}_r(\omega_o) = \frac{k_d}{N} \sum_{n=1}^{N} L_i(\omega_n)$$

Expressly stated probably

Importance Sampling in Practice

Case 2 (glossy BRDF)

$$f_r(\omega_n, \omega_o) = \frac{s+2}{2\pi} k_s \cos^s \theta_r$$

Isolate glossy component

Then, the estimator can be written as:

$$\hat{L}_r(\omega_o) = \frac{(s+2)}{2\pi} \frac{k_s}{N} \sum_{n=1}^{N} \frac{L_i(\omega_n) \cos^s \theta_r \cos \theta_n}{p(\omega_n)}$$

And we typically make cosine-based importance sampling taking:

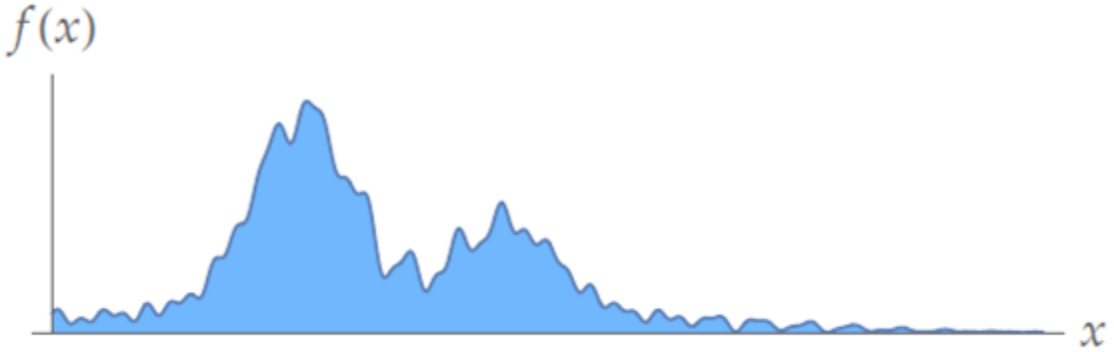
$$p(\omega_r) = \frac{s+1}{2\pi} \cos^s \theta_r \rightarrow \frac{s+2}{s+1} \frac{k_s}{N} \sum_{n=1}^{N} L_i(\omega_n) \cos \theta_n$$





Results: Uniform VS Importance Sampling

Link to Machine Learning



For complex multimodal integrands, the shape of the integrand might be dominated by the shape of $L_i(\omega_i)$

In this case BRDF sampling might not be enough

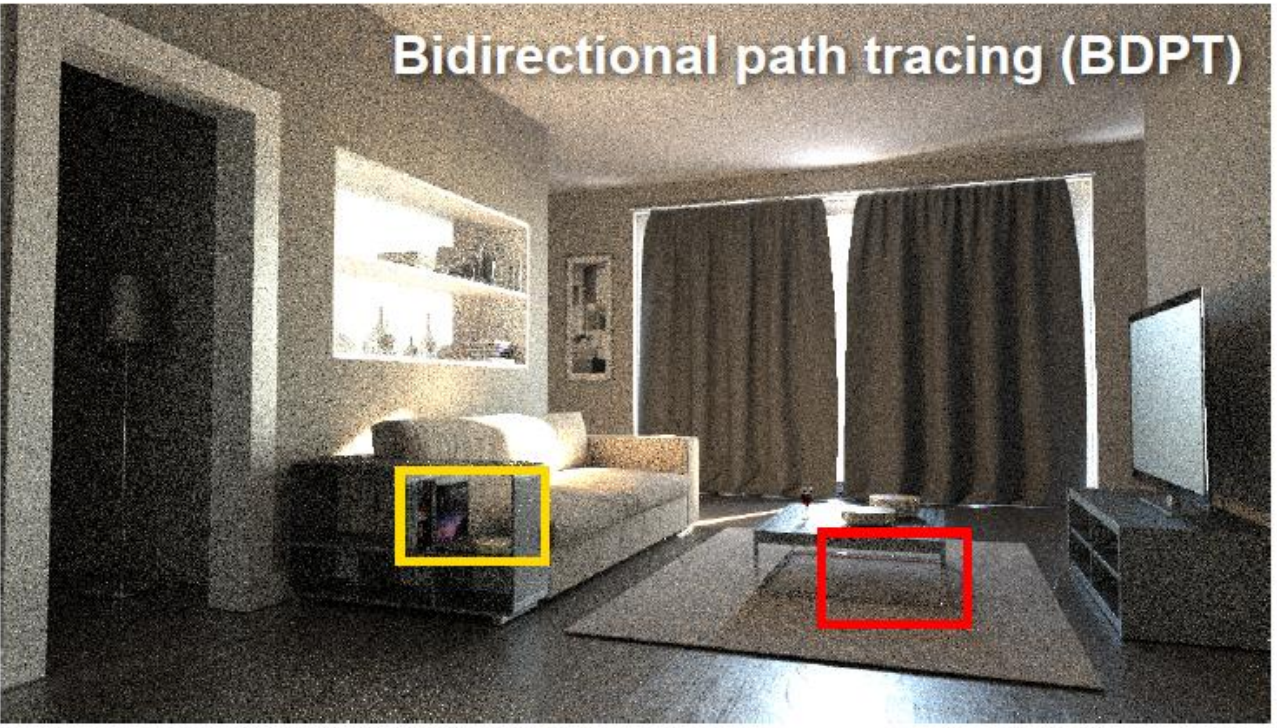
$$L_r(\omega_o) = \int_{\Omega(x)} L_i(\omega_i) f_r(\omega_i, \omega_o) \cos\theta_i \, \mathrm{d}\omega_i$$

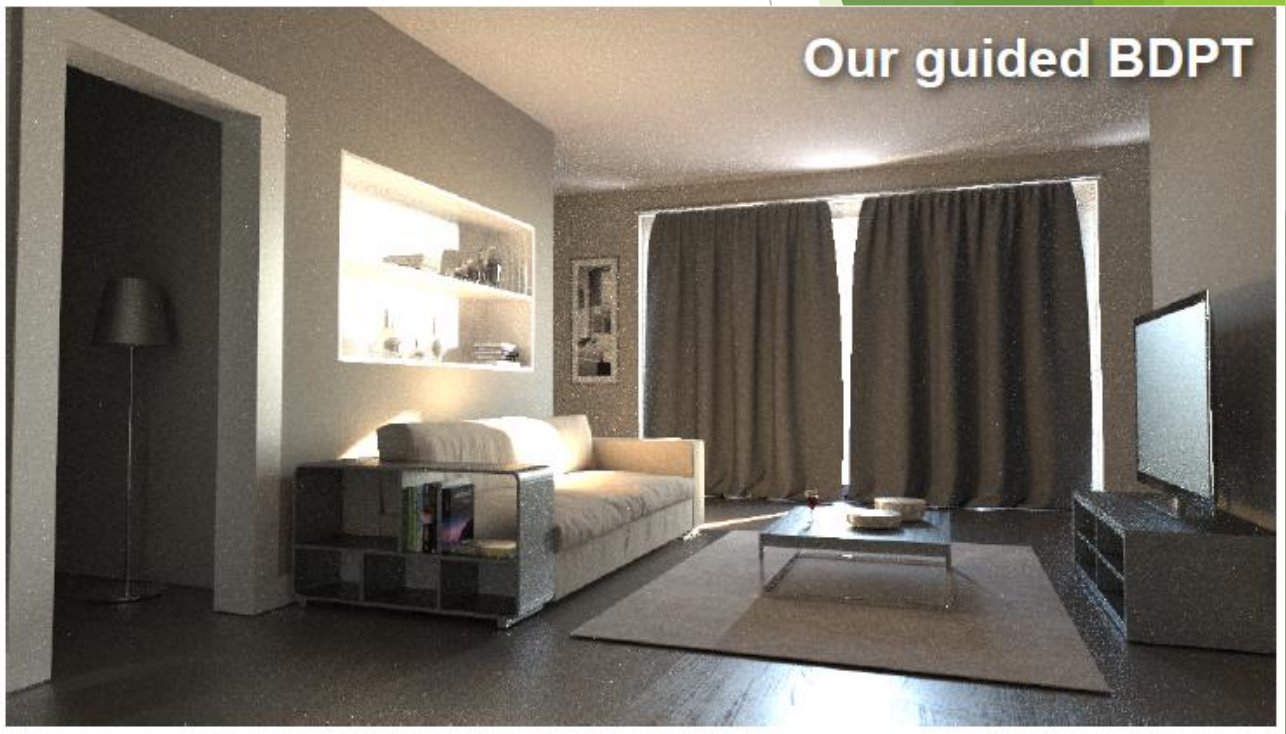
Many works have focused on learning “better” pdfs from a set of integrand samples

- “better” -> which contains information about the incident radiance function

Follow some examples (that you might choose as your paper to present)

On-line Learning of Parametric Mixture Models for Light Transport Simulation (2014)



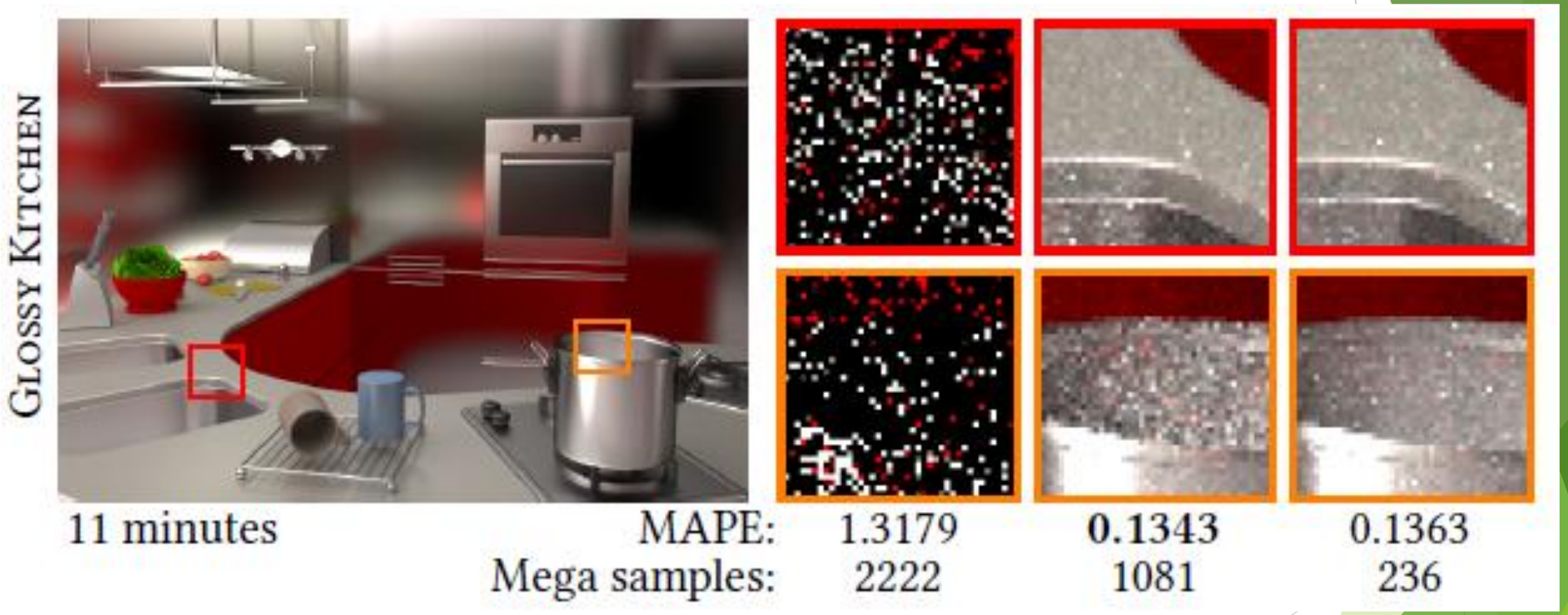


Product Importance Sampling for Light Transport Path Guiding (2016)





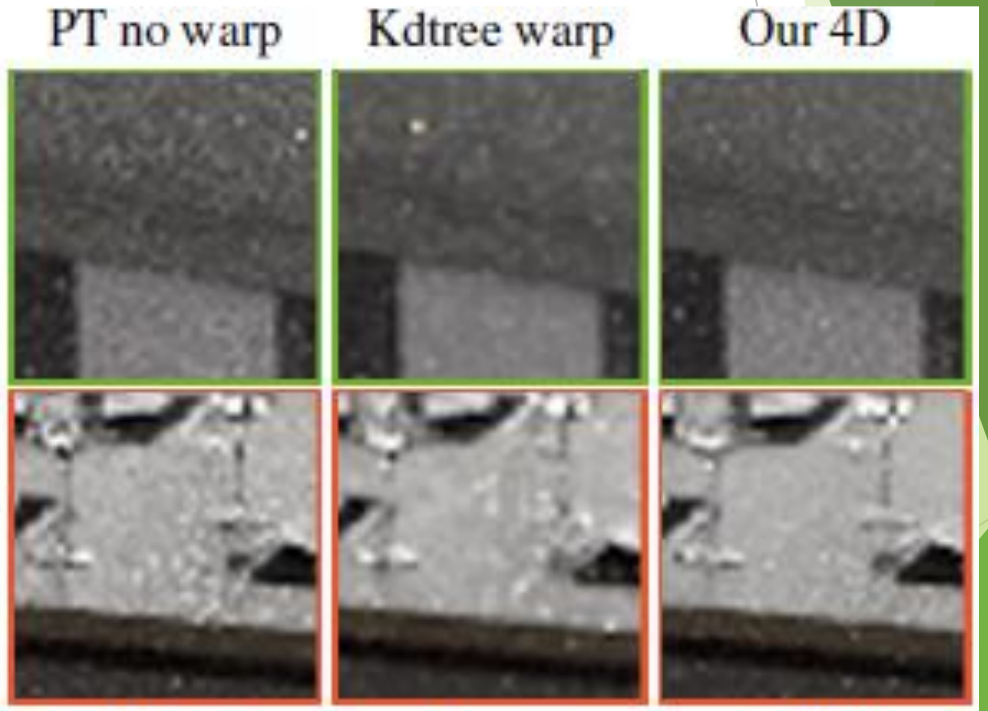
Neural Importance Sampling (2019)



Learning to Importance Sample in Primary Sample Space



COUNTRY KITCHEN



MSE

0.0401 0.0220 0.0170

Conclusion

In a Nutshell

- Path Tracing: simple to implement but costly to execute
- Large variance which decreases in slowly with the number of samples
- Clever sampling can help decreasing computational cost
  - Area light sampling for direct illumination
  - Importance sampling for indirect illumination
  - Can be coupled with Machine Learning techniques

Calendar

Week no.	Date	Theory	Practice	Seminar	Deliveries
Week 1	28 Sep 02 Oct	**T1**: Introduction	**P1**: Introduction to A1.1	-	
Week 2	05 Oct 09 Oct	**T2**: Basic Ray Tracing		**S1**: Work on A1	
Week 3	12 Oct 16 Oct	**T3**: Ray Casting, Recursive Ray Tracing	**P2**: Introduction to A1.2		
Week 4	19 Oct 23 Oct	**T4**: Rendering Equation, Monte Carlo Integration and Direct Illumination **T5**: Indirect Illumination and Path Tracing		**S2**: Work on A1	**D.A1**
Week 5	26 Oct 30 Oct	**T6**: Volume Rendering: Absorption-Only Model	**P3**: Introduction to A2.1		
Week 6	02 Nov 06 Nov	**T7**: Volume Rendering: Emission, Absorption and Scattering Model **T8:** Neural Radiance Fields and Topics' Presentation		**S3**: Work on A2	
Week 7	09 Nov 13 Nov	**T9**: Midterm	**P4**: Introduction to A2.2		
Week 8	16 Nov 20 Nov		**P5**: Work on A2	**S4**: Work on A2	**D.A2**
Week 9	23 Nov 27 Nov	**T10**: 1st Presentation	**P6**: Work on FP	-	
Week 10	30 Nov 04 Dec	**T11**: 2nd Presentation **(First Session)** **T12**: 2nd Presentation **(Second Session)**		**S5**: Work on FP	**D.FP**

Credits

- Thank you to Josep Blat for making available the slides for ACG (2023)
- Several slides and images are inspired on Berkeley's CS184/284A course on Computer Graphics



The End

Questions?